Client internals and version profiles, part 2 of 4

Client internals and version profiles, part 2 of 4

This part contains 20 notes, from `client-internals/live-gserver13.md` through `client-internals/feature-specs/README.md`.

`client-internals/live-gserver13.md`

The real 1.3 server and client, live under Wine (2026-10-30)

The 1999 `GServer13.exe` and the 1.3 test client `Graal13.exe` (both from the `graal13b` archive) were run together under Wine, inside a loopback-only network namespace. The traffic was captured with tcpdump. This is the first time an original server and its own client were played together in the lab. Tags: **[live]** seen in the running programs, **[cap]** read from the capture.

The raw capture, the decoded message list, screenshots, the run notes and the tools are in the project files under `graal-decompile/live-trace/early/`. This note has no addresses and no decompiler text.

Server settings and the default account [live]

GServer13 opens a start dialog before it listens:

setting	default
port	14804
respawn time (bushes etc.)	10 s
horse lifetime	10 s
alignment increment time	1200 s
baddies respawn time	60 s

These values are saved to `serverconfig.txt` as `serverport`, `respawntime`, `horselifetime`, `apincrementtime` and `baddyrespawntime`.

A new account from Edit accounts starts with these values:

- level `onlinestartlocal.graal` at x 30.5, y 30
- AP 50, MP 0
- 3 hearts of 3
- 0 rupees, 10 bombs, 5 arrows
- glove 1, bomb 1, bow 1
- sword 1 `sword1.gif`, shield 1 `shield1.gif`
- head `head0.gif`, colors orange, white, blue, red, black
- weapons enabled, no spin attack

The rights levels are no rights, Graalian Police, Trusted Admin and Administrator. The 1.3 zip does not ship `onlinestartlocal.graal`. It was copied from the 15th Graal build, which uses the same `GR-V1.03` level format.

Login [cap]

- **Client's first frame.** u16 big-endian length + zlib of id 0 + `GSERV014` + length-prefixed account + length-prefixed password + `\n`. Lengths are chr(n + 32).
- **Wrong account.** The server sends id 16 with the text "An account with the name and password you typed in doesn't exist.", and the client offers Reconnect or Play offline.
- **Good login, server order.**
    1. 9: player props.
    2. 14: warp, with x, y and the level name.
    3. 100: raw-data size, 8194.
    4. A **separate** zlib frame holding 101 (board) + 8192 bytes. In 2.05 the size message and the board share one frame.
    5. 6: level name.
    6. 1: level links.
    7. 5: signs.
    8. 39: level modification time.
    9. 3: NPC props, one message per NPC, carrying the whole script.
    10. 1.
- **Client's next messages.** It asks for each NPC image with 23, sets flags from the NPC scripts with 18, and adds horses with 7 (from `puthorse`). The server answers 30 (file not sent) for images it lacks. It deletes each horse with 18 about 9.5 to 9.8 s later, which matches the 10 s horse lifetime on a 1 s server tick.
- **Server sweeps.** The horse deletes and the bush regrows all landed on the same fractional second, so the server checks its timers once a second.

In play [live] [cap]

- **Walking.** While a key is held, the client sends one id 2 (player props) message every 100 ms, carrying X (prop 15) and SPRITE (17). Speed is 0.5 tile per 50 ms tick, the same as 2.05.
- **Chat.** Sent as id 2 prop 12. It is drawn as plain white text with a blue outline above the head, with **no bubble** (`shots/chat13.png`).
- **Cutting a bush.**
    - The client sends id 1 (board modify) with x, y, w = 2, h = 2 and the four leftover tiles.
    - The server does not echo the change back to the cutter.
    - The server regrows the bush with id 7 and the original tiles 2 3 / 18 19.
    - Two cuts regrew after 7.6 s and 9.2 s. Both fit the 6 to 12 s formula in `board-respawn.md`.
    - A bush cut also dropped rupees, which were rolled on the client, as that note describes.

Wine notes for 1.x clients

- **Keyboard.** The 1.3 client gives DirectInput a keyboard data format that lists one object, with a 256-byte buffer. Windows filled the whole buffer. Wine fills only the listed objects, so the arrow keys never reach the game. Keys do reach DirectInput, and chat and Alt shortcuts work, so the player just stands still.
    - Patching the exe on disk trips the client's own checksum ("The Graal execution file is corrupted").
    - `tools/kbfix.py` in the project files instead rewrites the format in memory to a full 256-key table right after start, before the client sets it. Walking then works.
    - Other Delphi builds that use the same DirectX header probably need the same fix at their own address.
- **Display.** Run a window manager (openbox) on the Xvfb display. Set Wine's Direct3D renderer to `gdi`. With the GL renderer, each DirectDraw blit was very slow.

- **Settings.** Edit `graalconfig.txt` (`gamemode=One Player Online`, `serveraddr=<loopback>`) rather than typing into the server combo box, which mangled the text.

Which builds start under Wine [live]

Each archive was unpacked and its client started offline for 15 s (`survey-all-builds.txt` in the project files).

- **Opens its window:** every build from the 1st to 15th Graal builds (including the 4th, 5th and 9th alphas), 1.1, 1.1.9, 1.28b2, 1.3 test, 1.31 to 1.39, 1.41.1, 2.0 (all revs), 2.01, 2.03, 2.04, 2.05 (all revs), 2.1 test 2 and 3, NewMain 24 and 26, NewWorld beta 1 and 2, and Graal 3D.
- **Exits at once:** 2.02, all three revisions.
- **NewMain 24:** opens a "File not found" box.
- **No client exe in the archive:** 1.21 and Zelda Online 2.

`client-internals/versions-133-139.md`

Client versions 1.33 to 1.39

Date: 2026-10-23, live check 2026-10-30

This note covers what makes each of the seven gen-2 GNW clients different, both on the wire and for the player. It is the writeup for `version-profiles/v133-v139.toml`, the machine-readable table that a client such as Seihai can load to behave as one exact build. The table gives every key in full for every version. This note only describes changes.

It is cleaned: no addresses, function names or decompiler output. The builds are the ones in `build-catalogue.md`, identified by MD5 there, and every shipped `Graal.exe` matched its catalogued hash.

Sources and method

What	How it was read	Confidence
Server → client ids, prop ids, UDP ids, GS1 command tables	Jump tables and the command table, as described in `cross-version-timeline.md`	confirmed
Client → server ids	Every call to the TCP send helper with a constant id. The helper is the one that prefixes `chr(id+32)`, appends `\n` and flushes at 4096 bytes.	confirmed for constant ids. A few call sites pass the id in a register and are not counted.
Login	The code that builds the first message after connect, read in 1.33 and 1.39	confirmed in 1.33 and 1.39. 1.34 to 1.38 are assumed to be the same.
Windows, options, menus	The Delphi form resources (captions and components)	confirmed
`graalconfig.txt` keys	Plain strings ending in `=`	confirmed. From 1.38 on some strings are built one character at a time, so those lists may be short.
HUD and tiles	`state.png`, `sprites.png` and `pics1.png` shipped beside each exe, compared tile by tile	confirmed
Player-facing behaviour	The shipped `newfeaturesNNN.txt` and `readme.txt`	confirmed as release notes, not checked in code
Server behaviour	GServer-v2 source (public). Its version checks only distinguish 1.41 and 2.1+, so it treats all of 1.33–1.39 the same. Checked live against GServer-v2 beta4, see "Live check".	confirmed live for 1.33 and 1.39

What	How it was read	Confidence
Live capture	1.33 and 1.39 run under Wine against a local GServer-v2 beta4, with the traffic captured and decoded. See "Live check" below.	confirmed for 1.33 and 1.39

What is the same in all seven

- **Wire.** Gen 2: every flush is one `u16be length` + zlib stream, with no size threshold, no cipher and no key byte. See `protocol/gserv-1999.md`.
- **Login.** Login is message id 0, which is `chr(32)` and also marks the client type as player. It carries the 8-byte token, then `chr(len+32)` + account, then `chr(len+32)` + password.
- **Server.** There is no listserver. The connect dialog is pre-filled with `<list-host-classic>:14900` and the `guest`/`guest` account. There is no server list window.
- **Levels.** Only GR-V1.00 to 1.03 load (`.graal`, plus `.zelda` Z3-V1 for the offline levels). There is no `.nw` and no `.gmap`. The board does arrive as `RAWDATA` (100, with an i3 length) followed by `BOARDPACKET` (101) in its own frame (confirmed live). Saves are written as `Graalsv11` (the reader also takes `Graalsav9` and `Graalsv10`; see `../formats/graalsav.md`).
- **Files.** The client asks for files with `WANTFILE` (23) and never sends `UPDATEFILE` (34). It handles no large-file ids. The reply is `RAWDATA` (100, i3 length) then `PLO_FILE` (102): `chr(len+32)` + name, then the file bytes, with no modtime (confirmed live in 1.39). For clients before 2.1, the server adds `.gif` to image names that have no extension.
- **No tampercheck** (39).
- **UDP.** There is a peer UDP channel for "data packages", with `noudp=` and `fixedudpport=` (default 14899).
- **Game modes.** One Player, One Player Online, and Two, Three or Four Players on one machine. Local multiplayer goes away in 2.0.
- **Window.** The window is 640×480 (648×508 outer). Alt+Enter switches to fullscreen.
- **Keys.** F1 opens options, F2/F3 load and save, F4 opens the editor, F5 loads a level, F6 opens the debugger, F7 opens the player list, and Alt+1 to Alt+7 take screenshots, show the minimap, show the bytes counter, take a snapshot and record.
- **Prop encodings.** These are not traced per build. Live, the server-sent forms of props 1–9, 11 and 13 parsed in 1.33 and 1.39 as the pre-2.1 forms in `protocol/player-props.md`. The rest are assumed (*inferred*). The forms are: sword `c(power+30)+str`, shield `c(power+10)+str`, bow `c(10+len)+image` or `c(power)`, head `c(len+100)+image`, and X/Y in half-tiles. No build in this range has X2/Y2. Prop 10 must be the bow form; a server that sends a gani string there (GServer-v2 beta4 does) is misread, and the rest of that prop stream is lost.

Protocol changes

Build	Client → server	Server → client	Player props	Other-player props	NPC props	UDP
1.33	0–31	0–41, 100	0–33 *[inf]*	0, 2, 8–13, 15–21, 24, 30–32, 50, 51	0–34 *[inf]*	0, 10–23
1.34	same	same	0–33	+ 34	0–34	same
1.35–1.37	same	same	same	same	same	same
1.38	+ 32 `ITEMTAKE`, 33 `WEAPONADD`; − 25	+ 43 `DEFAULTWEAPON`	same	same	same	+ 24
1.39	+ 80 `PROFILEGET`, 81 `PROFILESET`	+ 75 `PROFILE`	+ 34 `ACCOUNTNAME`	+ 33	same	same

Some of these protocol steps line up with release notes.

- **1.34 adds other-player prop 34 (account name).** The same release saves ignores by account name (`ignoredaccounts.txt`).
- **1.38 adds `ITEMTAKE`.** The 1.38 notes say "when you get the item then the server increases your rupee count", which is the anti-dupe change.
- **1.38 changes how weapons are registered.** Up to 1.37 the client uploads each weapon it holds with id 25 (name, image and the full script, in the same layout as `PLO_NPCWEAPONADD`). From 1.38 it sends id 33 as `chr(0)` + `chr(default weapon id)` instead, and the server answers default weapons with 43 `DEFAULTWEAPON`. Both confirmed live (25 in 1.33, 33 in 1.39).
- **1.39 adds the profile window**, with request (80), set (81) and reply (75).

So a server can only tell 1.38 and 1.39 apart from the earlier builds by these ids or by the token. gs2lib also knows `GNW02070` (1.38 rev1) and `GNW12080` (1.39 rev1). Neither build is in the archive.

What the player sees change

HUD art (`state.png`, `sprites.png`)

- **1.33 and 1.34.** `state.png` is 224×81, and the three item buttons carry the letters **A**, **S** and **D**. The HUD digits 0–9 sit at the bottom of `sprites.png` (128×484).
- **1.35.** A small **CHAT** label is added to `state.png`. It marks the focused player in 2/3/4-player mode, a feature the 1.34 notes describe.
- **1.38.** The button letters are removed from the art, because keys became configurable in that release. The client still draws A/S/D on the buttons itself (seen live in 1.39). The digits move from `sprites.png` (now 128×452) into `state.png` (now 224×97).

A client that copies these looks needs the A/S/D button art up to 1.37, and blank button art with the letters drawn on top from 1.38. It also has to read the digits from the right sheet.

Tiles (`pics1.png`, always 1856×512)

The tileset is unchanged from 1.33 to 1.37. 1.38 changes 81 tiles ("some new tiles in pics1.png") and 1.39 changes 7 ("added some tiles and corrected the tile type for the dirty water").

Windows and options

Build	Change
1.33	Player list (F7) with four sections (GPs, Guild, Normal, Ignore). Its popup menu has Message, Ignore and History. More Options has 13 checkboxes: vsync, joystick, music, sound, reverse stereo, save downloaded images, external chat, compatibility draw, hide decompression errors, fixed UDP port, no UDP, ignore mass messages, no DirectInput.
1.34	The F6 debugger is rebuilt with Next, Run, Stop next call and Stop next script, plus panes for this.vars, vars, flags and NPC attributes. It only works offline.
1.35	Player list popup gains **Buddy** (`buddyaccounts.txt`). More Options gains "Don't use HTML for PMs". PMs render as HTML. The editor gains "Generate map" *[inf]*. A "(Senior GP)" tag string appears.
1.36	More Options gains "Limit the length of nicknames to [50]". An "(RC)" tag string appears.

Build	Change
1.38	The editor gains a "Show blocking tiles" checkbox. "Support for external chat programs" and its in/out message files are removed.
1.39	Player list gains **Profile** and **Options** buttons and a Profile popup item. There is a new Player List Options window (ignore mass messages, no HTML, no images in PMs, sort by nickname). There is a new Profile window (real name, age, sex, country, ICQ, e-mail, homepage, favourite hangout, favourite quote). More Options swaps the moved PM options for "Don't connect levels", "Ignore toalls" and "Don't load heads for the playerlist / map".

Controls

- **Up to 1.37** the keys are fixed: arrows or IJKL, D item, S or Space attack, A action, M map, Q item list, P pause, Tab chat.
- **From 1.38** each action can have up to three keys, set in `graalconfig.txt`. The actions are `up`, `down`, `left`, `right`, `item`, `attack`, `action`, `map`, `switchtochat`, `itemlist` and `pause`. The usable key names are A–Z, 0–9, Tab, Enter, Shift, Space, P_Up, P_Down, the arrows, N0–N9, Add, Sub, Mul, Div, Dec, and F9–F15.
- **1.38** adds Alt+8 and Alt+9 for zoom, and Ctrl+letter emotion icons from `levels/emotions` for A, B, D, E, F, G, H, L, M, O, P, R, S, T, W, X and Z. AFK, BRB and Zzz stay until the player moves.
- **1.39** adds Ctrl+C, K and Y. It also adds chat-line history on Up and Down (100 lines), and lets the player click the weapon list to select.

Gameplay and rules (release notes)

- **1.34.** Hitting a mounted player costs their horse one bush, and at zero bushes the rider is pushed off. Two players can share a horse offline. Sounds are limited to one per type per frame and eight at once. `canbepulled` is separate from `canbepushed`.
- **1.36.** The server cap rises from 220 to 28000 players. Scripts can read the level object arrays (`bombs[]`, `arrows[]`, `items[]`, `explos[]`, `horses[]`).
- **1.37.** Sound volume falls off with distance. `showimg` works on normal NPCs. `playlooped` and `stopsound` are added. Arrays can be initialised as literals, and `true`, `false` and `hasweapon()` are added.
- **1.38.** A nickname equal to the account name shows as `*nick`. Item pickup is checked by the server. A laid-down shield is visible to others. Scripts can read and write `npcs[]`, and `hitnpc`, `setlevel2`, `changeimgpart` and the `test*()` functions are added. Explosions are capped at 1024. The `gotbow`/`gotsword` flags are dropped. PMs are saved to `pms/`.
- **1.39.** Head images are handled differently. Dead players don't block. Weapon-list and M-map fixes. In the editor, tiles can be painted by dragging, and an image dropped on the level becomes an NPC.

Script commands

The GS1 command table grows from 88 to 93, 93, 104, 106, 109 and 111 commands. The full lists are in the TOML, and the per-build additions are in `cross-version-timeline.md`. Two of the 1.34 additions, `carryobject` and `throwcarry`, are only announced in the 1.37 notes. So being in the table does not prove a command works in that build.

Other shipped files

- **Languages.** English, Deutsch and Portugues up to 1.36. Español and Italiano are added in 1.37, Swedish in 1.38 and Français in 1.39.

- **World maps.** The world map image grows from 616×784 (1.33) to 952×952 (1.36) and 1008×952 (1.37). `mappw` is added in 1.35 and `mapmantara` in 1.37. These are server content, not client features.
- **gif2mng** ships from 1.38.

Using the table

- `[[version]]` holds the full values for one build. `[version.features]` holds that build's on/off flags, and `[version.assets]` describes which HUD and tileset revision it ships.
- The top-level `[features]` table gives each flag's first build, its confidence and a one-line note. `[confidence]` gives one line per key on how sure each value is.
- To pick ids, use `pli_sent` for what the client may send, and `plo_known`, `plprops_known` and `npcprops_known` for what it parses. A server that sends anything else to one of these builds hits the default branch, and the client ignores it. Confirmed live in 1.33 and 1.39 for ids 47, 82, 156, 168, 190 and 194.

Live check (2026-10-30)

1.33 and 1.39 were run under Wine against a local GServer-v2 beta4 with a local listserver copy, on loopback only, and the traffic was decoded.

- **Login props.** After the level loads, the first id 2 carries props 0 (nick), 11 (head), 13 (colours), 18 (status), 31 (UDP port) and 33 (additional flags), the same in both builds. The head goes as `chr(head number)` when it is a numbered default head. 1.34 to 1.38 are assumed to match.
- **Steady traffic.** Id 31 `PACKETCOUNT` goes about every 10 s. Id 3 NPC props go for each level NPC after the load, and level scripts cause id 18 and 7.
- **Board and files.** As described under "What is the same in all seven". For a name sent without an extension the server adds `.gif`. Downloaded images were not written to disk in this setup, and the level was cached under `weblevels/<server>/`.
- **Server message.** 1.33 shows the server message (41) as raw HTML text.

Server compatibility (GServer-v2 beta4)

- It needs `generation = classic`, with the GNW tokens listed in `allowedversions.txt`. Otherwise it refuses the client with "Allowed: 2.0 - 2.31".
- The server checks the account with a listserver, even though the client connects directly. Without one the client is told "The login server is offline".
- It kicks 1.33 for id 25 ("Unknown Player Packet: 25"). 1.34 to 1.37 send the same id (*inferred* to be kicked too). A proxy that drops id 25 lets 1.33 play, though beta4 then warns about an invalid packet count, because the dropped messages are missing from the count.
- 1.39 logs in and plays. 1.38 has the same send set apart from profiles (*inferred* to work).
- It sends prop 10 as a gani string, which these clients misread (see "Prop encodings").

Open

- Login props in 1.34 to 1.38 (seen live only in 1.33 and 1.39).
- Per-prop byte layouts for props not seen live (assumed above).
- The client → server UDP id set. The receive set is confirmed.
- The exact letter-to-icon mapping for emotions. The names are built at run time, and only AFK (A), BRB (B) and Zzz (Z) are given in the notes.

`version-profiles/matrix/rules-139.md`

1.39 death, hurt, knockback, sword reach and wall test

Date: 2026-10-31. Closes matrix unknown #1 for **1.39** (era C). Tags: **[bin]** from the 1.39 client (plain behaviour only; no addresses), **[lab]** / **[1.39]** from the offline Wine mannerisms run already in `../../client-internals/player-mannerisms-live.md`. Tick = 50 ms.

Compared with the 2.05 rules in `../../client-internals/decomp-findings-2026-10-24/01-death-respawn.md`, `../../client-internals/decomp-findings-2026-10-24-d/01-sword-damage.md` and `../../client-internals/decomp-findings-2026-10-25-k/01-knockback-and-packet-hits.md`.

Summary table

Rule	1.39	2.05 (for contrast)	Tag
Hurt mode length	40 ticks; cannot be hurt again while in mode 3	same	**[bin] [lab]**
Knockback	1 tile per tick while the hurt counter reads 39 down to 35 (five steps); zero vector becomes one tile east; twelve body-box probes; blocked step undone	same five steps	**[bin]** (earlier live table said four tiles; see below)
Hurt pose / blink	pose ~10 ticks then normal; blink starts after the pose, every other tick to 40	`hurt` gani 31 ticks; blink from the hit	**[lab]**
Sword reach	0–4 tiles ahead of the attacker in the facing; sideways offset under 2 tiles	same	**[bin]**
Sword swing length	counter 1→6 (5 ticks); cuts bush	5 ticks (`sword.gani`)	**[lab]**
Death timer	counts 0→30 and stops at 30	same	**[bin]**
Input while dead	any movement/action key respawns once the timer is above 27	same (after 27)	**[bin]**
STATUS dead bit (0x08)	set only when life-state is dead **and** the timer is above 27	same (≥ 28)	**[bin]**
Online default respawn spot	x 30.5, y 30.0 in the current level when no saved entry	same	**[bin]**
Respawn hearts	shared heart-write used on respawn restores to **max hearts**; no AP-band test in this path (1.39 has no AP HUD; bands arrive with 1.41)	AP bands (3 / 5 / max)	**[bin]**
Wall / walk test	six probe offsets per facing step; blocked count pushes at 18 and wraps 38→18; walk 0.5 tile/tick	six probes; same push counters; same speed	**[bin] [lab]**
Baddy touch power	0.5 hearts (1.0 for type 7); within 1 tile on both axes of the baddy centre	same family of powers in 2.05	**[bin]**

Hurt and knockback [bin]

A hit is refused when the player is already in hurt mode (mode 3), in mode 7, or not in the playing life-state. An accepted hit:

1. Clears the current pose sprite.
2. Sets mode 3 and the hurt counter to **40**.
3. Applies the power to hearts (negative power heals; hearts stop at max).
4. Stores a knockback unit vector. A zero vector becomes **one tile east**.
5. If the player was carrying something, may force a drop when the power is positive.

Each playing tick while in mode 3: the counter drops by one; at 0 the mode returns to normal. While the counter is **strictly greater than 34** after that drop (readings 39…35), the player moves one tile along the stored vector, then twelve body-box points (four edges × three) are tested; if any is blocked the step is undone for that tick.

The offline Wine run (`player-mannerisms-live.md`) measured a **40-tick** no-rehit, a ~10-tick hurt pose, blink after the pose, and **four** tiles of knockback on the touch-NPC test. The binary's counter window is the same five-step window as 2.05; the four-tile live figure is kept as a lab observation that may have lost a step to collision or ±1 tick video error. Prefer the binary's five steps for a faithful profile unless a re-run shows otherwise.

Sword reach [bin] [lab]

When another player's swing is judged against the local player, a hit counts only if the victim lies in a facing-relative box: **0 to 4 tiles** ahead of the attacker's centre, and **strictly under 2 tiles** of sideways offset. Power and knockback then go through the shared hurt routine. Offline, S runs a 5-tick sword counter and cuts a bush in front on the first frames (`player-mannerisms-live.md`).

Death and respawn [bin]

When hearts fall to 0 or below:

1. Hearts are clamped to 0; the hurt counter is set to **0**; mode returns to normal; life-state becomes **dead** (value 2).
2. `dead.wav` plays; inventory is partly spilled as floor items (rupees in 100/30/5/1 bands, arrows and bombs similarly).
3. Each tick while dead: if the counter is under **30** it increments by one (so it stops at 30). Between counters **4 and 27** the client cycles death sprites. Above **29** a cleanup helper runs.
4. **STATUS bit 0x08** is included in the prop encoding only when life-state is dead **and** the counter is above **27**.
5. A movement or action key while dead with counter above **27** (or while in life-state 4) runs the respawn routine.

Respawn clears the hurt counter and modes, restores the shared heart field through the same write used when filling hearts to max, warps to the last saved level entry when one exists, and otherwise (online, no saved name) places the player at **x 30.5, y 30.0**. Offline without a saved entry uses the built-in start-level table. No separate "respawn" packet exists on this path.

Wall / walk test [bin] [lab]

Ordinary walking uses a facing step scaled by the move speed (0.5 tile per tick when unburdened; 0.2 when hidden; other multipliers for horse/shield). The candidate position is tested with **six** probe offsets from a facing

table against the tile/NPC block test. A failed probe refuses that step. The blocked-step counter that drives the push pose still uses the **18 / wrap-at-38 → 18** rule already measured live for 1.39. Sliding along a blocking NPC at 0.125 tile/tick was seen live; it is a consequence of the same probe path, not a separate rule.

Knockback reuses the block test with its own twelve-point body box (centre near X+1.5, Y+2.0), matching the later 2.05 layout in spirit.

What Seihai should change

See `seihai-gaps.md` (updated in the same change): for a 1.39 profile, use the 2.05 death timer and sword box, keep hurt lock-out at 40, prefer **five** knockback steps from the binary (not the earlier four-tile live cell), restore hearts to **max** on respawn (no AP bands), and keep the six-probe walk / 18–38 push counters already shared with later builds.

Still open

- Exact numeric contents of the six walk-probe and twelve knockback-probe tables (offsets only; behaviour above does not need them for Seihai's current box tests).
- A fresh Wine knockback distance re-measure to reconcile the five-step binary with the four-tile live cell.
- 1.32 and 1st Graal (next rows of matrix unknown #1).
- Online GServer13 capture of STATUS bit timing and CLAIMPKER (id 14) on a real death; not required to tag the client-side rules above.

`client-internals/cross-version-timeline.md`

Client cross-version timeline, 1.33 → 2.1 tests

Date: 2026-10-21

This note follows the Delphi client across twenty builds: 1.33 to 1.39, 1.41 rev1, NewMain 1.42, 2.0, 2.01, 2.02 rev2, 2.03, 2.04, 2.05 rev1, 2.05 rev2, the two 2.1 test builds and the two NewWorld betas. Builds are identified by MD5 in `build-catalogue.md`. Function data come from the same static analysis as `build-diff-204-205.md` (#11). All facts here come from RTTI, the GS1 command table and the packet switch tables, read the same way in every build. It is cleaned: no addresses, no decompiler output.

How it was read

- **Classes.** Class names, parents, instance sizes and virtual-slot counts come from the Delphi VMTs.
- **GS1 commands.** The script executor matches commands against one table of fixed-size records, each holding a pointer to the command text. The table was found by its first entry (`say`) and read to the end. Records are 16 bytes in 1.33 and 12 bytes from 1.34 on. Trailing spaces in a command (`say`, `setshape`) are part of the match text and mean the command takes arguments.
- **Packet ids.** Each packet and prop parser is a Delphi `case` on the first byte minus 32, compiled as a jump table (two-level in most parsers: a byte index table, then a pointer table). The ids that reach a real handler, not the default branch, are the ids the build handles. Ids are listed after the minus-32 step, so they are protocol ids.

Function maps onto 2.05 rev2

Every build was matched function by function onto 2.05 rev2 (6529 functions) with the matcher described in `build-diff-204-205.md` (#11), applying the 2.05 virtual-slot shift for the level-object classes. The per-build maps (old function, rev2 function, name, identical or changed) are kept outside the repo with the lab files, since they carry addresses.

Build	Functions	Identical to 2.05 rev2	Changed	Only in this build	Only in 2.05 rev2
1.33	5228	4312	345	571	1872
1.34	5240	4299	388	553	1842
1.35	5435	4479	383	573	1667
1.36	5437	4477	385	575	1667
1.37	5459	4494	395	570	1640
1.38	5546	4534	439	573	1556
1.39	5593	4550	470	573	1509
1.41 rev1	5675	4783	419	473	1327
NewMain 1.42	6468	5592	507	369	430
2.0	6396	5637	515	244	377
2.01	6412	5663	528	221	338
GraalX (2.01 with `.nw`)	6429	5662	531	236	336
2.02 rev2	6350	5678	484	188	367
2.03	6428	5754	479	195	296
2.04	6471	5799	558	114	172
2.05 rev1	6522	6345	94	83	90
NewWorld beta 1	6381	5810	405	166	314
2.1 test (first)	6598	5774	611	213	144
2.1 test (second)	6595	5762	619	214	148
NewWorld beta 2	6484	5668	489	327	372

Most of the identical count for the 1.3x builds is shared Delphi library code. The largest single step is 1.41 rev1 to NewMain 1.42, about 800 more functions, part of which is the new DirectDraw 7 and grid library code. GraalX (see `build-catalogue.md`) matches 2.01 in its classes, GS1 commands and packet ids, so it is left out of the tables below. Against 2.01 it differs only by the `.nw` reader and writer. The 2.1 tests have more functions than 2.05 but fewer that are unique to rev2, which fits them being a later branch of the 2.05 code.

Classes

Build	Added	Removed
1.34	`TServerPlayer`, `TEditorBaddy`	`TOtherClient`, `TEditorOpponent`, the OLE classes (the new names look like renames of the removed ones)
1.35	`TWebBrowser` and the OLE control classes	—
1.36 to 1.38	—	—
1.39	`TPlayerListOptions`, `TProfileEdit`	—
1.41 rev1	`TGraalClient`, `TServerListClient`, `TServerListFrame`, `TServerMessage`, `TShortMessage`, `TWinamp`	`TAdventureBoolOperation`, `TRTFMessage`
NewMain 1.42	`TGraalAni` (+ `Part`, `Sound`, `Sprite`, `Step`), `TGraalImage`, the DelphiX DirectDraw 7 classes, string and draw grids, `TClipboard`	`TAdventureSign`

Build	Added	Removed
2.0	—	the DDE client/server classes
2.01	—	—
2.02 rev2	—	the zlib error classes (back in 2.03)
2.03	`TNPCMove`	—
2.04	`TLevelObject`, `TObjectSet`, `TTempTile`, `TTileDef`	—
2.05 rev1	`TGraalMap`, `TADP` (anti-debug)	`TEditorBaddy`, `TEditorLevel`
2.05 rev2	—	—

Against 2.05 rev2:

Build	Added	Removed
NewWorld beta 1	`TLevelHeights`, `TAdventureBacktileArea`, `TDrawText`	horse, ball and carry objects, `TComputer`, `TEditorChest`, `TChestEdit`, `TEditBaddy`, `TPos`, `TWebClient`
2.1 test (both)	`TEditNPC`, `TEditVariable`, `TEditorDefNPC`, `TPageControl` / `TTabSheet`, `TMemo`; the second test adds `TSmilie`	`TPersonAction`, `TEditPerson`, `TShowCodes`, `TObjectSet`
NewWorld beta 2	both lists above, including `TSmilie`	both lists above

So **NewWorld beta 1 was cut from the 2.05 code base and beta 2 from the 2.1 test code base.** Beta 2 carries every 2.1-test class change on top of the NewWorld removals.

Sizes of the main game classes (bytes, virtual slots)

Class	1.33	1.34	1.38	1.39	1.41	NM 1.42	2.0	2.01	2.02r2	2.03	2.04	2.05	NW b1	2.1 test	NW b2
`TAdventure` (main form)	3268	3240	3244	3256	3268	1864	1864	1860	1860	1860	1084	1084	1084	1108	1108
`TAdventureLevel`	56	84	84	84	84	84	88	88	88	88	96	184	33968	184	33968

Class	1.33	1.34	1.38	1.39	1.41	NM 1.42	2.0	2.01	2.02r2	2.03	2.04	2.05	NW b1	2.1 test	NW b2
`TAdventureCommand`	64	68	76	76	68	68	68	68	68	68	68	76	76	76	76
`TDrawableObject`	20	20	20	20 / 2	28 / 2	28 / 2	28 / 2	28 / 2	28 / 2	28 / 2	32 / 2	32 / 3	32 / 3	44 / 3	44 / 3
`TServerPlayer`	—	220	224	240 / 2	272 / 2	272 / 2	272 / 2	280 / 2	288 / 2	288 / 2	340 / 7	336 / 8	320 / 8	364 / 9	348 / 11
`TPlayer`	1516	1512	1524	1544	1564	1496	1500	1516	1528	1528	804	800	760	832 / 836	792
`TEditorNPC`	208	220	228	224	244	476	476	488	496	508	568	564	548	592	572
`TGraalMap`	—	—	—	—	—	—	—	—	—	—	—	76	96	256	34060

Two points for anyone carrying names between builds by virtual-slot number:

- **2.04 puts `TLevelObject` under the player and NPC classes**, which is why `TServerPlayer` jumps from 2 to 7 slots. Slot numbers from 2.03 and earlier do not line up with 2.04 for these classes.
- **2.05 adds a slot to `TDrawableObject`**, shifting every descendant by one (see `build-diff-204-205.md`, #11). The 2.1 tests and NewWorld beta 2 add more slots to the player classes again.

The NewWorld `TAdventureLevel` (33968 bytes) and beta 2 `TGraalMap` (34060 bytes) are large enough to hold tile arrays inline, which fits the NewWorld level heights and back-tile layers.

GS1 commands

Count of commands in the table, and what each build adds over the previous one. The table is followed directly by a short direction table (`up`, `right`), and in some builds by other keyword lists (`*=`, `:=`, the 1.33 soldier names, and `nwweek` / `nwmonth` / `nwmin` in the NewWorld betas). Those entries are not counted. In 2.05 this gives 161 commands, matching the earlier count on main.

Build	Count	Added
1.33	88	—
1.34	93	`dontblocklocal`, `blockagainlocal`, `showlocal`, `hidelocal`, `carryobject`, `throwcarry`, `setbacktile`; `seteffect` and `unseteffect` leave the ta in NewMain 1.42)
1.35	93	same table as 1.34
1.36	104	`lay2`, `take2`, `takehorse`, `takeplayerhorse`, `removebomb`, `removearrow`, `removeitem`, `removeexplo`, `removehorse`, `explodebomb`, `reflectarrow` arguments
1.37	106	`playlooped`, `stopsound`

Build	Count	Added
1.38	109	`setlevel2`, `changeimgpart`, `hitnpc`; `setlevel` now takes arguments
1.39	111	`setani`, `setcharani`
1.41rev1	126	`canwarp`, `callnpc`, `say2`, `break`, `continue`, `callweapon`, `spyfire`, `function`, `return`, `changeimgvis`, `setimg`, `setimgpart`, `drawovertrees`, `put` `showstats`; `putexplosion` now takes arguments
NewMain1.42	134	`canwarp2`, `setshape`, `seteffect`, `setcoloreffect`, `setzoomeffect`, `drawaslight`, `changeimgcolors`, `changeimgzoom`
2.0	142	`setgender`, `setchargender`, `join`, `freezeplayer2`, `unfreezeplayer`, `enabledefmovement`, `disabledefmovement`, `setmusicvolume`
2.01	144	`openurl2`, `hitobjects`; `openurl` now takes arguments
2.02rev2	146	`tokenize`, `tokenize2`
2.03	148	`triggeraction`, `move`
2.04	157	`addtiledef`, `addtiledef2`, `removetiledefs`, `replaceani`, `attachplayertoobj`, `detachplayer`, `setshape2`, `enableselectweapons`, `disablese` `setshape` now takes arguments
2.05rev1/rev2	161	`setbacktile2`, `setz`, `updateterrain`, `loadmap`; `setbacktile` now takes arguments
2.1tests	161	same table as 2.05

Build	Count	Added
NewWorldbeta1	161	same table as 2.05
NewWorldbeta2	165	`addstring`, `insertstring`, `removestring`, `deletestring`

A command being in the table does not mean it does anything. #4 lists the 2.05 commands that have no executor case, so they parse and are ignored. Examples include `setz`, `updateterrain` and `setbacktile2` (2.05), `setbacktile` (1.34), `drawovertrees` and `canwarp` (1.41), `canwarp2` (1.42), `join`, `freezeplayer2` and `unfreezeplayer` (2.0), and `seturllevel`, `setfocus` and `resetfocus`. See #4 for the full list. This note has not checked executor cases in the other builds.

New commands are inserted in the middle of the table, not appended. So the internal index of a command changes between builds, and an executor decompiled in one build cannot reuse another build's case numbers.

Packet ids handled by the client

Server → client packets

Build	Change
1.33 to 1.37	43 ids: 0–41, 100
1.38	+ 43
1.39	+ 75 (45 ids: 0–41, 43, 75, 100)
1.41 rev1	same set
NewMain 1.42	+ 42, 44, 45, 150, 151, 153
2.0	+ 154, 155
2.01	+ 46, 156
2.02 rev2	+ 47
2.03	+ 48, 165
2.04	+ 166; − 26
2.05 rev1 / rev2	+ 168
NewWorld betas	2.05 minus 2, 4, 17, 18, 154, 155
2.1 tests	2.05 plus 49, 169

Id 42 is not handled until NewMain 1.42, although 43 is handled from 1.38 on. NewWorld drops six ids at once, alongside the removed baddy, horse and chest classes.

Prop parsers

Each prop switch reads a run of `(prop id, value)` pairs. Ids handled:

Parser	1.33	Added later
Player props	0–33 *[inf]*	34 (1.39); 35, 36 (1.41); 37–41 (2.02 rev2); 42 (2.04); 43–45 (2.1 tests)
NPC props	0–34 *[inf]*	35 (1.41); 36–40 (2.02 rev2); 41–43 (2.1 tests)
Baddy props	0–10	none; the switch is gone in both NewWorld betas

The player switch was told apart from the NPC switch by its callers: the player prop parser is the one that calls the sparse switch described below. That holds in every build from 1.34 on. In 1.33 neither switch calls it within its own body, so the 1.33 row assumes the same split as 1.34 (one switch is 0–33, the other 0–34). So NPC prop 34 is handled from 1.34 at the latest (from 1.33 if the assumed split holds), well before player prop 34 in 1.39.

The five ids added together in 2.02 rev2 (player 37–41, NPC 36–40) sit at the same place in both lists, so they are probably one feature added to players and NPCs at once. This is consistent with the public prop numbering used by open-source server work, where both lists gain five attribute props at these positions.

The player prop parser also calls a second, sparse switch with ids up to 51 (21 ids in 1.33, 23 in 1.39 and 31 in 2.05). It handles a subset of player props and grows in step with the player prop list. Its role (likely props of other players) has not been confirmed.

UDP packets

The UDP parser handles ids 0 and 10–23 in 1.33 to 1.37. 1.38 adds 24, 2.01 adds 25, 2.03 adds 26 and the 2.1 tests add 27. The NewWorld betas drop 16 and 17.

`client-internals/newworld-betas.md`

NewWorld beta clients (graalnwb1, graalnwb2)

Date: 2026-10-21

The two NewWorld betas are forks of mainline client builds:

Build	Linked	Code base	Compared against
graalnwb1 (`Graal-NW.exe`, MD5 88d1f84c…)	2001-08-06	2.05	2.05 rev2
graalnwb2 (`Graal-NW.exe`, MD5 af2d2106…)	2001-10-03	2.1 test	2.1 test 3

Both betas went through the same Ghidra pipeline as `build-diff-204-205.md` (Delphi compiler spec, RTTI names, switch-table recovery, feature export, `tools/build-diff/fdiff.py`). Each beta was compared with its own base, so what's left over is the NewWorld delta. Every change below appears in **both** betas unless the text says otherwise. This note is cleaned: it has no addresses and no decompiler output.

Pair	Functions (old / new)	Identical	Changed	Only old	Only new
2.05 rev2 → nwb1	6647 / 6494	5917	423	307	154
2.1 test 3 → nwb2	6703 / 6579	6051	354	298	174
nwb1 → nwb2	6494 / 6579	5715	611	168	253

The nwb1 → nwb2 row is mostly the 2.05 → 2.1 base change. The NewWorld-specific code is the same in both betas.

Classes

Added (both betas): `TLevelHeights`, `TAdventureBacktileArea`, `TDrawText`. Each is a plain `TObject` with no published methods.

Removed (both betas): `TComputer`, `TEditBaddy` (baddies), `TAdventureHorse` (horses), `TEditorChest`, `TChestEdit` (chests), `TAdventureCarry`, `TAdventureBall`, `TPos`, `TWebClient`. This matches `readmenw.txt`: "horses are npcs now", "there are no built-in horses, carry object, freezeplayer etc. anymore, see the control-npc script for it". Baddies and chests are scripted too.

All other class differences between the two betas (`TEditNPC`, `TEditVariable`, `TEditorDefNPC`, `TSmilie` and others) come from the 2.05 → 2.1 base change and are not NewWorld code.

Instance sizes:

Class	2.05 rev2	nwb1	2.1 test 3	nwb2
`TAdventureLevel`	184	**33968**	184	**33968**
`TGraalMap`	76	96	256 (now a `TAdventureLevel` subclass)	34060
`TServerPlayer`	336	320	364	348
`TPlayer`	800	760	836	792

The 33968-byte level: a 65 × 65 height grid

`TAdventureLevel` in the betas is a 168-byte header followed by a **65 × 65 grid of doubles** (65 × 65 × 8 = 33800; 168 + 33800 = 33968). The grid holds one height per tile corner of the 64 × 64 board, row-major with a stride of 65.

The 168-byte header has five fewer object lists than 2.05's (most likely the lists for the removed baddy, chest, horse and carry objects) and one new list, the backtile areas (see below). The other fields move to fill the gap. The board is still a separate `TBoard` (4096 tiles), as in 2.05.

In nwb2 the 2.1 base had already made `TGraalMap` a subclass of `TAdventureLevel`, so the gmap inherits the 33968-byte level part and adds its own 92 bytes.

Where the heights come from

Heights apply only when the current level is part of a gmap that has a height file, and only in one game mode (internal value 5; which mode that is wasn't confirmed). Otherwise every height reads as 0.

1. **Map corners.** When a gmap loads, the client looks for a sibling file named `<gmap base>_heights<ext>`, so `world.gmap` gives `world_heights.gmap`. (The shipped samples are named `..._heights.txt` next to

a `.txt` map, for example `levels/test3d/test3dmap_heights.txt`.) The file is a CSV grid of (width + 1) × (height + 1) quoted entries, one per level corner:

```
"100:36646","50:29374","100:97473","100"
"50:4646","-30:342","-50:12393","-50"
```

Each entry is `height` or `height:seed`. The seed in the entry at (x, y) belongs to the level whose top-left corner that is. The entry at the bottom-right corner (width, height) holds the map-wide seed.

2. **Level edges.** Each edge between two corners is filled by 1-D midpoint displacement out to 64 steps. The midpoint gets the average of the two ends plus `(random − 0.5) × 2 × amplitude`. The amplitude starts at 4.0 and is multiplied by 0.6 at each level of subdivision. The RTL random seed is set to the map-wide seed first. Neighbouring levels share their edges, so seams match.

3. **Level interior.** When a level is entered, its four 65-sample edges are copied from the map. The RTL random seed is set to that level's seed, and the interior is filled by recursive subdivision. Each step displaces the centre column and centre row, then recurses into the four quadrants. The amplitude starts at 4.0 × 0.6 and also shrinks by 0.6 per step. The terrain is deterministic: the same file always gives the same heights.

4. **Per-level control grid (`TLevelHeights`).** The gmap holds one `TLevelHeights` per level: an enabled flag and a 9 × 9 grid of doubles (one point every 8 tiles; 4 + 4 + 81 × 8 = 656 bytes). When the flag is set, each 8 × 8 cell gets a bilinear correction, so the generated surface passes exactly through the 81 control points.

    - nwb1 creates these objects but nothing in it sets the flag, so the control grid is inert there.
    - nwb2 fills them from a `HEIGHTS` … `HEIGHTSEND` block in the gmap parser (the generator form in `formats/grmap001.txt`). That block sets the flag.

Using and changing the heights

- Drawing players and other level objects looks up the heights of the four corners of the tile under the object.
- **`setz x,y,w,h,z1,z2,z3,z4`** writes heights into the grid itself. Every grid point in [x, x+w] × [y, y+h] gets a bilinear blend of the four corner values. A zero width or height averages the corners instead. Points outside 0..64 are skipped.
- **`updateterrain`** re-clamps the view and redraws the terrain surface. The `readmenw.txt` workflow is "change `waterheight` / `groundheights[]`, then call `updateterrain`". Those two variables are in nwb2 and the 2.1 base, not in nwb1.

In 2.05 itself `setz`, `updateterrain` and `setbacktile2` are parsed but have no executor case (see `build-diff-204-205.md`). The betas are the builds where they do something.

Backtile areas (`TAdventureBacktileArea`)

`setbacktile tile` sets one level-wide "back tile", the tile drawn behind trees and other foreground tiles. **`setbacktile2 tile,x,y,w,h`** adds a 24-byte area record (tile, x, y, width, height) to the level's area list and redraws the board. To find the back tile for (x, y), the client scans the areas in the order they were added and uses the first one whose rectangle contains the point. If none does, it uses the level-wide back tile. Both commands work on the current level in play and on the editor's level.

Deferred text (`TDrawText`)

While a frame is drawn, the client keeps a temporary list. Any nickname or chat text drawn during the scene pass is not drawn right away. It is queued as a 28-byte `TDrawText` holding the text, x and y as doubles (offset by the level's position in the gmap), and the colour. After the scene (terrain, objects, overlays) is finished, the queue is drawn in one final pass and freed. Outside a frame, text is drawn immediately as in 2.05. The likely purpose is to keep raised terrain and neighbouring levels from covering names. It matches the beta changelog line "Nicknames/ Chat texts are displayed in the correct level."

GS1 changes

The command table keeps the 2.05 layout: 12-byte records (name, a flags word, an argument-signature string). It has **161 commands in 2.05 and nwb1** (`say` … `hide`) and **165 in nwb2**. After it come a 4-name direction list (`up`, `left`, `down`, `right`), then, in both betas only, an **8-name NW time-variable list**:

Variable	Range (from `readmenw.txt`)
`nwday`	1…28
`nwweek`	1…40
`nwyear`	1000…
`nwweekday`	1…7
`nwmonth`	1…10
`nwtime`	0…1439 (minutes)
`nwhour`	0…23
`nwmin`	0…59

These are script variables (also usable as `#v(name)` message codes), **not commands**. The calendar has 28-day months, 10 months and 40 weeks. The server sends the time (packet 42), and "2000/5/1 realtime was 1000/1/1 graal time".

String-list commands (nwb2 only): `addstring`, `insertstring`, `removestring` and `deletestring` are inserted after `loadmap`, with the signatures shown here:

Command	Arguments
`addstring list,text`	string, string
`insertstring list,index,text`	string, number, string
`removestring list,text`	string, string
`deletestring list,index`	string, number

They go with `sarraylen(list)`, `lindexof(list,text)` and `#I(list,i)`, which are also only in nwb2. Lists are stored as normal string flags in comma form (`numbers="12 34",567,"8,9"`). nwb1 has none of this. Its `readmenw.txt` doesn't mention string arrays either; the section appears only in nwb2's copy.

Executor cases. These counts compare the betas with their bases. They are the same for both pairs, apart from nwb2's four string commands.

- Live in the betas, ignored in the base: `setfocus`, `resetfocus`, `setbacktile`, `drawovertrees`, `setbacktile2`, `setz`, `updateterrain`.
- Parsed but ignored in the betas, live in the base: `removecompus`, `putcomp`, `putnewcomp`, `hitcompu` (baddies); `puthorse`, `takehorse`, `takeplayerhorse`, `removehorse` (horses); `carryobject`, `throwcarry`, `takeplayercarry`; `setsword`, `hidesword`, `setbow`; `freezeplayer`; `enabledefmovement`,

`disabledefmovement`; `toinventory`; `followplayer`. The control NPC script (`controlnpc.txt`) takes over movement, swords, carrying and horses.

Packets

Server → client (TCP)

The receive switch covers 0…168 in 2.05 and 0…169 in 2.1. Compared with its base, each beta **drops six receivers**:

Id	2.05 meaning	Why it goes
2	BADDYPROPS	no built-in baddies
4	LEVELCHEST	no built-in chests
17	HORSEADD	horses are NPCs
18	HORSEDEL	horses are NPCs
154	FREEZE	no built-in freeze; `freezeplayer` is gone too
155	UNFREEZE	as above

Everything else in the base's receive set is still there. nwb2 also has ids 49 and 169, but they come from the 2.1 base.

Client → server (TCP)

Several 2.05 senders have no matching function in nwb1: HORSEADD (7), BADDYPROPS (15), BADDYHURT (16) and OPENCHEST (20). The bomb, arrow, item-add and item-take senders match 2.05 exactly. "No match" is an upper bound (see the matching caveat in `build-diff-204-205.md`), but it agrees with the removed classes.

Peer-to-peer UDP

The UDP receive switch covers 0…26 in 2.05 and 0…27 in 2.1. Both betas drop **16 and 17**, the horse add/remove pair that mirrors TCP 7/8 (see `protocol/udp-p2p.md`). They keep 0 and 10–15 and 18 up to the end of the range. nwb2 keeps the 2.1 addition, 27.

List server

Both betas hide the list-server host with the same DES string as 2.05 (`<list-host-classic>`; see `des-and-hidden-strings.md`). They build the port and list name one character at a time, as 2.05 does:

Build	Port	List name sent with id 0
2.05 rev2	14921	`newmain`
nwb1	14921	`newworld`
2.1 test 2/3	14922	`newmain`
nwb2	14922	`newworld`

So the port comes from the code base (14921 in the 2.05 line, 14922 from the 2.1 tests on), and the list name is the only NewWorld change. The betas log in to game servers with the token `G2N15041` instead of the mainline `GNW03081` (see `build-catalogue.md`).

Live lab runs (2026-10-30)

Both betas were run under Wine (32-bit prefix, virtual X display) in a network namespace whose hosts file points the GraalOnline names at a local stub. The stub answers as list server (14921 to 14923) and as game server, and logs every packet. The 2.1 test 3 build was run the same way for comparison. The per-version results are in `version-profiles/141-217.toml` (`vnwb1`, `vnwb2`).

List server

Build	Port	What it sends
nwb1	14921	id 0 `newworld` only
nwb2	14922	id 0 `newworld`, then id 1 with account and password (two GSTRINGs)
2.1 test 3	14922	id 0 `newmain`, then id 1 with account and password

So sending the account to the list server begins with the 2.1 tests (September 2001), not 2.17. nwb1, on the 2.05 code, does not do it. nwb2's start window is GraalOnline-themed with Account and Password fields. In one run it sent 23 characters of a 28-character password; the cut-off was not pinned down.

Game server login

Both betas send client type 0, the key byte, `G2N15041`, the account and the password, then LANGUAGE (37) `English`. Their first props are the same six as 2.05: NICK (0), HEADGIF (11), COLORS (13), STATUS (18), UDPPORT (31) and ADDITFLAGS (33).

- **COLORS has seven bytes**, the same in every run, where 2.05 has five. Both betas' prop parsers read eight colour bytes. A server must take the length from the prop and must not echo seven bytes to a client that reads a different count.
- HEADGIF comes without an extension (`headalgo000`).
- STATUS has a new bit, 0x20.
- The first login sometimes stalled with the level drawn but no HUD or player and no props sent. Logging in again fixed it every time.

After joining

- **Files.** nwb1 asks WANTFILE `shield1.png` and UPDATEFILE `bodynw.png` and `timenums.png`. nwb2 also asks for `idle.gani` and `headalgo000`. Neither asks for the sword, bomb and bow images 2.05 asks for.
- **Movement.** GANI (10) goes out as a string. While idle the client re-sends `idle` about every 100 ms, because the control NPC calls `setani` each tick (2.05 sends it only now and then). Walking sends X and Y in half tiles plus SPRITE, and nwb2 re-sends `walk` on each step.
- **Keys.** S does nothing online, so a server has to give the sword as an NPC weapon. A plays the grab animation. The control NPC comes from the local `controlnpc.txt` as the weapon `Control-NPC`, added when the player is created.
- **Time (packet 42).** The payload is a GINT4: minutes since 1000/01/01 00:00 on the NW calendar (28-day months, 10 months, 280-day year). A value of 162310450 showed as 1402/06/16 14:10 on the HUD clock.
- **Offline Training** in nwb1 runs without a server (HUD 150/150, clock 1000/01/01).

Client → server set

A scan of the send sites (same method as `version-profiles/README.md`) shows both betas drop seven ids that their bases send: 7, 8, 11, 15, 16, 17 and 20 (horse add/delete, throw carried, baddy props/hurt/add, open chest). That widens the four listed under Packets above. The prop parsers and writer otherwise match each base, and there is no baddy-props parser.

Corrections to other notes

- `build-catalogue.md` credited "string arrays" to nwb1. They are nwb2-only. nwb1's distinctive features are the terrain heights, backtile areas, deferred text and NW time variables.
- The 2.05 GS1 table has 161 commands. A count of 163 reads two entries past the end (`up` and `right` from the direction list that follows).

`client-internals/v2.00-connect-path.md`

2.00: list reply, Connect and the grey window

Date: 2026-10-30. Build: Graal v2.00 (`GNW24011`, md5 `c93e659e2cda36379aaea4706db20b38`). Static reading of the exe, checked against the cross-laptop lab of 2026-10-31 (seihai `documentation/cross-laptop-2026-10-31.md`). Derived facts and addresses only. Tags as in `protocol/listserver.md`: **[bin]** · **[cap]**.

The answer

The lab 2.00 run that never reached the gserver was not a list-format problem. The client folder (a copy of the 2.22 distro) had no `objects.dat`. 2.00 checks for it when Start is pressed, sets a fatal flag, and the Connect path later returns early on that flag. The result is a blank grey game window, no socket and no DNS lookup. **[bin]** The lab saw exactly that. **[cap]** The lab listserver and GServer-v2 need no change for 2.00.

Classes

Delphi RTTI, VMT addresses: `TGraalClient` 0x4dabc8 (parent TObject), `TServerListClient` 0x4dac4c, `TAdventureClient` 0x4dacbc. `TServerListClient` overrides virtual slot 2 (0x4db984, the per-message handler) and slot 3 (0x4dbc3c, an empty `ret`). Slot 0 (0x4db160) closes the socket. **[bin]**

List reply [bin]

- 0x4db984 acts on ids 0 and 1 only. Any other id, including the 2 (status), 3 and 5 the lab listserver sends, is ignored.
- Id 0: `GCHAR count`, then per server `GCHAR field count` and that many `GCHAR len + text` fields, each server kept as a string list. The list goes to the list form (0x4a4ffc), and the handler then closes the list socket through slot 0. A row is drawn only with at least 6 fields (0x4a512c), and the detail view needs at least 8 (0x4a5250).
- Id 1: the same field layout for **one** server, appended to the current list (form field +0x330). If it is then the only row, it is selected. This is an "add one server" message, not a login answer.
- So the lab's single frame (id 0 list, id 2 status) and the earlier two-frame 2.17-style reply are both accepted.

Connect [bin]

1. Connect button (0x4a604c): needs non-empty account and password, keeps them, then calls 0x4a61e0.
2. 0x4a61e0: the selected server must have at least 8 fields. **Field 6 is the host and field 7 is the port**, copied to the globals at 0x5397a4 and 0x5397a8. Fields 8–15 are optional (two strings and two numbers when there are at least 12, the same again for 16). Then 0x52a458 and **0x525c0c** run, and the game form is shown (0x44aa60).
3. 0x525c0c: **returns at once if the byte at 0x53a610 is set.** Otherwise it lays out the game window, then creates `TPlayer` (0x50ca0c).
4. `TPlayer` constructor: loads `letters.png` and `head26.gif`, does `SetCurrentDirectory(<game dir>levels)`, and when the host global is non-empty creates `TAdventureClient` (0x4dbc40).
5. 0x4dbc40: if the host or port is empty it shows "The connection could not be established!" without dialling. Otherwise it calls 0x4db17c: `WSAStartup` once, a name cache (0x4dad94), `inet_addr`, then `gethostbyname`, `socket`, `TCP_NODELAY`, `connect` to `htons(StrToInt(port))`, non-blocking. Any failure raises, and the same message is shown.

The lab list (name, language, description, url, version, players, ip, port) puts the IP and port where 2.00 reads them.

The fatal flag

0x53a610 is set only on a fatal error, right after `Application.MessageBox` and before `Application.Terminate` (0x44da90, which is `PostQuitMessage`). It is never cleared. The message is "File not found" / "The file X has not been found in the Graal folder!". **[bin]** The sites are in the asset loader 0x52525c (`sprites.png`, `pics1.png`, `state.png`, `opps.png`, `bomb1.png`, `bomb2.png`, and `objects.dat` via 0x52d36c → 0x52d3c1 `FileExists`), plus `arrays.dat` (0x52330d) and some quit paths.

0x52525c runs from the main menu's Start handler (0x4a78d0), before the server list opens. With `objects.dat` missing, the flag is set there. The list window still opens and works, but Connect stops at step 3. In the lab no dialog was seen after Connect. **[cap]** Whether the "objects.dat" box appeared at Start, or was hidden behind the menu, was not recorded.

`head26.gif` was also missing from the lab folder, but `TPlayer` loads it through `TSurfaceOwner` (VMT 0x4f3ca0, load at slot +8, 0x4f3d28), which has its own exception handler. It is not fatal.

Lab rule for 2.00

Run 2.00 from a folder that has its own `objects.dat` (the 2.22 distro has none), plus `arrays.dat`, `letters.png`, `sprites.png`, `pics1.png`, `state.png`, `opps.png`, `bomb1.png` and `bomb2.png`. The earlier lab run that reached the gserver used the 2.22 folder plus an `objects.dat` (`decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`).

Confirmed [cap], 2026-10-30: after `objects.dat` from NewMain 26 (2001-01-15, 33852 bytes) was copied into the same folder, a re-run with the same lab servers showed no dialog at Start or Connect. 2.00 opened TCP to the gserver on 14802, drew the server's start level and chatted. Nothing else in the folder or on the servers changed.

`client-internals/build-diff-204-205.md`

2.04 → 2.05 → 2.05 rev2: function-level diff

Date: 2026-10-21

This compares the three builds closest to the decompile target. All three were run through the same Ghidra pipeline: import with the Borland Delphi compiler spec, apply RTTI-recovered class and method names, recover the switch tables, then export per-function features. The functions were then matched across builds. The function counts below come from this pipeline (Ghidra 11.4.2 with the tools in `tools/build-diff/`). Other Ghidra passes over the same binaries can report different totals, depending on how many switch tables and RTTI methods they recover. This note is cleaned: it has no addresses and no decompiler output. Builds are identified by MD5 in `build-catalogue.md`.

> **Live debugging:** 2.05 and every later build start an anti-debug component that looks for debuggers, loaders and monitors and stops the client when it finds one. 2.04 and earlier do not have it. See Anti-debug component.

How the matching works

Each function is reduced to an address-free instruction stream (mnemonics and operands, with in-image addresses replaced by a placeholder). The matcher then runs these passes in order:

1. Unique RTTI names (`Class.Method`, `Class.virt_N`, `Class.dyn_N`). Virtual-slot names are skipped for classes whose VMT shifted (see below).
2. Unique exact streams of 6 or more instructions.
3. Unique sets of referenced string literals.
4. Unique sets of large immediates.
5. Propagation through ordered callee lists and single unmatched callers, repeated until stable.

A matched pair is **identical** if the streams are equal and **changed** otherwise. Unmatched functions are counted as only-old or only-new. Small look-alike helpers (many 15-instruction VCL thunks, for example) can fail to pair, so the only-old and only-new counts are an upper bound on real churn.

Pair	Functions (old / new)	Identical	Changed	Only old	Only new
2.05 rev1 → rev2	6522 / 6529	6345	94	83	90
2.04 → 2.05 rev2	6471 / 6529	5799	558	114	172

2.05 rev1 → rev2

The game code barely moves. About 20 game functions differ. Class layouts are identical (same instance sizes, same VMT lengths) and the GS1 command table is the same. What changed matches the rev2 changelog:

- **Player counting** (changelog: "the script variable playerscount is fixed"). Four helpers count players in the current level and its gmap neighbourhood. In rev1 they read the level's player list through the caller's object for one test and through the global level for the other. rev2 reads both through the same object.
- **Tutorial / map start-up.** rev1 created a map object while setting up the tutorial map files. rev2 leaves that slot empty at start-up. When a gmap is active, rev2 runs an extra step after the map object is created.
- **Level reset.** One level-reset path no longer sets a flag that depended on game mode 5. A per-level loop takes its bound from a field on the main form instead of a hard-coded 632. These may belong to the changelog's "switching to other servers" fix, but that link is not proven.

- **NPC script text.** Of the three helpers that translate the § line marker into CR/LF, one was dropped and the other two were reworked.
- **Small changes** in the server-message window, the server-list form constructor, the main form constructor and the mouse-move handler (a few instructions each).

The library code was rebuilt. The VCL and RTL units in rev2 (Forms, ComCtrls, Controls, docking, ImgList and others) differ from rev1 in instruction shape. rev1's library code, by contrast, matches 2.04's. So rev2 was linked against a rebuilt or updated VCL, not just recompiled game units. For carrying names, this means RTTI-named VCL methods still match by name, but unnamed library helpers between rev1 and rev2 need the shape or callee passes.

2.04 → 2.05

This is where the NewWorld code lands in the mainline.

New classes: `TGraalMap` (gmap support) and `TADP` (see the anti-debug section below). **Removed classes:** `TEditorBaddy` and `TEditorLevel`.

The level-object VMT shifted by one slot. `TDrawableObject` gained a virtual method (2 → 3 user slots). Every descendant's virtual numbering therefore moves by one from 2.04 to 2.05: `TDrawableSurface`, `TSurfaceOwner`, `TLevelObject`, `TServerPlayer → TPlayer` / `TEditorNPC`, `TComputer`, `TDisplayedNPCImg`, `TEditorChest`, and the ten level-object classes (horse, bomb, ball, carry, extra, flying, leap, explosion…). So 2.04 `X.virt_N` corresponds to 2.05 `X.virt_(N+1)`. Any name or type carried between 2.0–2.04 and 2.05 by slot number has to apply this offset.

Instance-size changes: `TAdventureLevel` grows from 96 to 184 bytes (gmap and `.nw` state), `TAdventureCommand` from 68 to 76, and `TServerPlayer` / `TPlayer` / `TEditorNPC` each shrink by 4.

`.nw` support (GLEVNW01) is new code. The text-level reader parses `BOARD`, `LINK`, `BADDY …` `BADDYEND` and `CHEST` sections and decodes tiles with the standard base64 alphabet. It sits next to the binary `GR-V1.00–1.03` reader, which stays. A matching writer emits `GLEVNW01`, then one `BOARD 0 <y> 64 0 <tiles>` line per row (the `BOARD x y width layer tiles` form in `formats/glevnw01.txt`). A new chest-line parser carries the error "Wrong chest data!". Three other existing functions gained a GLEVNW01 magic check: one in the level class, one in the player/script class, and one in the main form unit. `.nw` is added to the level open dialog.

gmap support is new code. It covers the `TGraalMap` class, a loader that builds the map image (`map.png`) and is reachable from the GS1 executor, and a `bigmap.txt` / `minimap.txt` setup routine reachable from the reconnect window. A main-form routine that set up `bigmap.txt`, `minimap.txt` and `map.png` in 2.04 no longer references them.

GS1 commands new in 2.05: `loadmap`, `setz`, `updateterrain`, `setbacktile2`, and a `setbacktile` form that takes an argument. The 2.05 table holds 161 commands (163 records counting the `up` / `right` direction-table entries that follow it), identical in rev1 and rev2. Only `loadmap` does anything: `setz`, `updateterrain` and `setbacktile2` have no executor case in 2.05, so they parse and are ignored (see the parsed-but-ignored table from #4).

Smaller additions: - PMs are written to `pms\messages_<name>.txt`. - One drawing routine gains a `drawaslight` check. The `drawaslight` script command itself predates 2.0 (it is already in the NewMain 1.42 command table). - The script editor gains a 400-instruction routine that handles `/* */` comments and tabs. It is reached from a new button handler, matching the changelog's "new button in the script editor". - A new level-editor routine, reached from a toolbar button, handles `playerx` / `playery` with values clamped to 0–61. - 2.04's

routine that installed `tempsitc.ttf` into the Windows fonts folder was not matched to anything called in 2.05. The string and the code body are still in rev2, so it looks uncalled rather than removed. The `Courier New` default in the main form constructor is gone.

Anti-debug component (ADP), new in 2.05

2.05 links a third-party Delphi component `TADP`. The main form constructor starts it, and the form's DFM sets all four switches: `Kill_Debugers = True`, `Stop_Execution = True`, `Anti_Procdump = True` and `Kill_Monitors = True`. It uses the Toolhelp32 process, module and heap walkers. It looks for loader and monitor tools by name (the binary contains `FPLOADER.EXE` and `FROGSI`, likely a fragment of a FrogsICE-style name) and stops execution when it finds one. The component appears in 2.05, 2.05 rev1, 2.05 rev2, the 2.1 test builds and both NewWorld betas. It is absent from every build up to 2.04. This matters for live work on 2.05 and later under a debugger or monitor.

What to reuse

- **2.05 rev1 is close enough to share names with rev2** wherever the RTTI or shape match holds. Only the ~20 functions above, plus library helpers, need re-checking.
- **About 90% of 2.04's functions are identical in 2.05.** It is the cleanest reference for what the NewWorld merge added: TGraalMap, GLEVNW01, the VMT slot, the new GS1 commands and ADP.

`client-internals/feature-timeline.md`

Client feature timeline, 2.00 to 2.31

Date: 2026-10-21

Source: the release notes shipped in the Graal Reborn 2.2.2.0 folder (`newfeatures2001.txt` … `newfeatures2005.txt`, summarized). The file names are release years, not version numbers: `2001` covers 2.00–2.12 (it also repeats 1.32–1.41), `2002` covers 2.13–2.17, `2003` covers 2.18–2.21, `2004` covers 2.22, and `2005` covers 2.3–2.31. The protocol column links each note to what the binaries show (`protocol/client-versions.md`). Tags: **[doc]** release notes · **[bin]** client binaries.

Version	Release-note highlights	Protocol anchor
2.00	Online-only: 2/3/4-player mode and save games removed. Bush rupees and pot hearts go server-side. `warpto <account>` for GPs.	
2.01	Level preloading on maps. `hitobjects` added as the fair replacement for `hitplayer`. `openurl2`.	`PLI/PLO_HITOBJECTS` (36/46)
2.02	Hats and signs with images, string parsing. Free-server limits on light effects and "complex" ganis.	
2.03	`triggeraction x,y,action,params`. RC and NC get `/` chat commands. The built-in zlib.dll is replaced by in-game compression code "because it has been abused".	`PLI/PLO_TRIGGERACTION` (38/48)
2.04	Engine work: gani `PLAYSOUND PARAM1`, `replacegani`, sound and animation changes.	
2.05, 2.05 rev 2	Mostly bug fixes. Server name in the window title. **2.05 rev 2 is the `GNW03081` build.**	type 0, gen 3, 59 PLO ids **[bin]**

Version	Release-note highlights	Protocol anchor
2.1 (rev1–3)	New start window and server list with account info. New image engine without GDI. Gani scripts (`SCRIPT … SCRIPTEND`). String arrays. `play2`. Auto-mapping. **Big map and minimap set by the gserver (`bigmap=` / `minimap=` in serveroptions) instead of the listserver. Ghost mode** for free players, queued until activated. Alt+0 log window. `graalversion`.	`PLO_BIGMAP` / `MINIMAP` (171/172) and ghost ids (170, 173, 174) are handled by 2.17 but not 2.05 **[bin]**
2.12 (rev1)	Graal2001/Graal2002 client compatibility. Multi-level editing. `wraptext`, `wraptext2`.	
2.13 (rev3–4)	Editor control-NPCs. **New RC with file browser, server list, and RC+NC in one program.**	`RemoteControl.exe` (see `tools/remotecontrol.md`)
2.14 (rev4–7)	**Server list gets two tabs** (company servers, classic and playerworlds). Voice chat. **Player-list status icons** (`playerlisticons=` in serveroptions). **`serverwarp <world>`**. **Files over 64 KB can be downloaded** (`downloadpos`). Gmaps auto-download.	`PLO_STATUSLIST` (180), `SERVERWARP` (178), `LARGEFILESTART/END` (68/69), all handled from 2.17 **[bin]**
2.15 (rev2)	Bug fixes, "enabling Graal2002". `showimg2`.	
2.16 (rev1–7)	Real transparency (`seteffectmode`). Gani attributes `#P10`–`#P30`. More server-list features. UDP check improved, with a warning when UDP is blocked.	`PLPROP_GATTRIB10–30` (54–74)
2.17 (rev1)	Graal Kingdoms speedups. **Level heights stored in level files instead of the gmap.**	`PLO_BOARDHEIGHTS` (185) **[bin]**; still type 0, gen 3; adds `PLI_TAMPERCHECK` (39) and the signed `arrays2.dat` listserver config **[bin]**
2.18 (rev1)	**"New password encryption"** for stored passwords. Preloaded levels on entering a server.	
2.19 (rev1)	Released alongside v3. New start screen and server list. **"Better compression"** (bz2). `this.` strings on the client. On NPC-server servers the client stops sending non-`client.` strings.	gs2lib: `GNW15053`, type 4 / gen 4 (bz2 + partial XOR) **[srv]**
2.2	Multi-byte sign text (Korean, Chinese, Japanese). **Server-side tile changes on gmaps via `updateboard2`.**	`PLO_BOARDMODIFY2` (186), handled from 2.22 **[bin]**
2.21	**"Network traffic reduced drastically"**. `triggeraction 0,0,clientside,…` reaches a weapon's client-side part.	gs2lib: `GNW01113`, the compression byte **[srv]**
2.22	**Third server-list tab** for under-construction playerworlds (gold/VIP only). `gravity`. `downloadsize`.	type 5, gen 5, `GNW03014`, listserver `GNW30123` **[bin]**
2.3	**Editor and online client split into separate exes.** `FILENAMECACHE.txt`. New timer (no 99% CPU). B-tree NPC lookup. **Classic client-side mode removed: Classic moved to the NPC-Server.** Graal3D server tab. `graal://` URL handler with `serverstartparams`. **Full-precision player and NPC positions** ("enabled once all clients are switched"). `timevar2`. More hacking checks.	`PLO_HASNPCSERVER` (44) dropped, `PLO_MOVE2` (189) and `LARGEFILESIZE` (84) added in 2.31 **[bin]**
2.31	Float-to-string fix, keyboard config saved, gmap NPC border crossing fixed.	`GNW28015` **[bin]**

Reading the timeline against the binaries

- Several features announced as "will be enabled later" show up in a client's receive switch one or more versions before the server used them (full-precision moves in 2.3, level heights in 2.17). A handled id is therefore not proof that servers of that era sent it. **[bin][inf]**
- The jump from 79 to 81 handled ids between 2.17 and 2.31 is small. Most 2.19–2.31 changes are cipher, compression, and scripting changes, not new packets. **[bin]**

`client-internals/live-trace-205.md`

Live traces of the 2.05 rev 2 client

Date: 2026-10-29. These are behaviours seen in the running 2.05 rev 2 client: in Wine, offline with no network, with the player state read every 50 ms tick from outside the process. Hearts reports come from the 2.05 lab capture (`tools/lab-setup.md`, run 1) and from a loopback stub server. Tags: **[live]** seen in the running client, **[cap]** seen in a lab capture. This note has no addresses and no decompiler text. The harness, its scripts and the raw traces are in the project files under `graal-decompile/live-trace/`.

Hearts are reported by the client

- **On a hit, the client sends its own CURPOWER. [cap]**
  - In run 1, the player's own bomb hit them. In the same tick as the hit, the client sent one `PLI_PLAYERPROPS` message with CURPOWER = 5 half-hearts (down from 6), GANI `hurt` and SPRITE. Its knockback position updates followed.
  - Nothing from the server asked for it, and the server did not echo CURPOWER back.
  - The 2.22 lab shows the same thing (`lab-captures-2026-10-25/`): a hit sends CURPOWER + `hurt`; death sends the `dead` gani + STATUS; respawn sends CURPOWER by AP bracket.
- A client that applies damage locally but never sends CURPOWER leaves the server holding stale (full) hearts for that player.
- Not yet captured for 2.05: lava burns, heart pickups and respawn.
- Hearts are stored obfuscated in the client (`feature-specs/04-death-respawn.md`). A memory scan does not find them as a plain value. **[live]**

Ledge jump and running in place [live] (`new3.nw`)

- Walking up into the up-ledge at y = 14: the refused steps are counted 1 to 17, and the jump starts on the 18th.
- Jump y values from y0 = 14: 12.5, 11.3, 10.2, 9.5, 9.0, 8.8, 8.8, 9.0. This is exactly the curve in `ledges-push-pull.md`.
- **Running in place.** On every refused step the walk animation frame keeps advancing, and the gani stays `walk`. It changes only when the push gani takes over (at 18, against a wall) or the jump starts (at 18, at a ledge).
- While jumping the gani stays `walk`, and the blocked-step counter counts down 7 to 0 as the jump timer.

Water and lava [live] (`new4.nw`)

tile	what the client does
3644, dark-blue deep water	Blocks like a wall. `push` comes at the 18th refused step.
2882, bright-red lava	Swim: the gani becomes `lava`. Burns.

tile	what the client does
2958, dark-red lava floor	Walkable: the gani stays `walk` / `idle`. Burns.

- **Burn timing.** On either red tile a per-player counter runs 0 to 100 and wraps to 0 on the next tick. One cycle is therefore **101 ticks**, and each wrap costs half a heart. Three wraps took 3 → 1.5 hearts.
- Stepping onto grass resets the counter to 0.
- The liquid test uses the point (x + 1.5, y + 2).
- Burning to 0 hearts offline shows the `End` death scroll.

Server CURPOWER 0, death and respawn [live] (online, 2026-10-29)

The real 2.05 rev 2 client was logged in to a loopback stub server: login at (30, 30) on `onlinestartlocal.nw`, AP not sent. The player walked to (25, 42) before each death.

1. **The server sends CURPOWER 0.** The client at once sends GANI `dead`, and does not send CURPOWER. About 1.35 s later (27 ticks) it sends STATUS 0x1C, which sets the dead bit 0x08.
2. **A key after the timer.** The client sends `PLI_LEVELWARP` (0) to the same level at (30, 30). That is the login point, not the death spot. In the same frame it sends CURPOWER 6, GANI `idle`, X 30, Y 30 and STATUS 0x14 (dead bit cleared). It does not wait for a server reply.
3. **MAXPOWER 0 first.** If the server sends MAXPOWER 0 before the kill, the respawn still reports CURPOWER 6. The client does not cap its respawn hearts at a MAXPOWER of 0.
4. **CURPOWER 0 after each respawn.** The client dies and respawns the same way every cycle. A server that answers every revive with CURPOWER 0 (for example an account at MAXHP 0, which GServer-v2 clips to) puts a real client into a death loop.

`version-profiles/2.22-2.31.md`

Version profiles: 2.22, 2.3 and 2.31

Date: 2026-10-22

This note covers what a client has to do to *be* 2.22, 2.3 or 2.31: what it sends and understands on the wire, and how it looks and behaves for the player. The machine-readable table is `2.22-2.31.toml`.

Sources: - `Graal_222.exe` and `Graal_231.exe` from the Graal Reborn 2.2.2.0 folder, read statically. That covers form resources, embedded images, strings, the GS1 symbol table, and the send and receive paths. - The seihai-lab captures (2.22 only). - GServer-v2 source. - The release notes shipped in the same folder.

No 2.3 executable was uploaded, so every 2.3 value is inferred.

Tags: **[bin]** client binary · **[cap]** lab capture · **[srv]** GServer-v2 · **[doc]** release notes · **[inf]** inferred.

Builds in this range

Label	FileVersion	Token	Binary	GServer-v2 default `allowedversions.txt`
2.22	2.2.2.0	`GNW03014`	yes	enabled
2.3	2.3.0.0	`GNW14015`	**none**	commented out
2.31	2.3.1.0	`GNW28015`	yes	commented out

A lab server needs `GNW14015` and `GNW28015` uncommented before it will let 2.3 or 2.31 in. **[srv]**

Caveat on the binaries: the Reborn folder's `Graal_222.exe` has a 2009 file date and `Graal_231.exe` a 2007 one. The 2.17 build in the same folder is rebranded "Eradia Online", so a community edit of the other two cannot be ruled out. Any edits would most likely be to text and server-list strings, not to the protocol code. **[bin][inf]**

Wire protocol

The three builds share the gen-5 profile already written up in `protocol/client-versions.md` and `protocol/codec-and-framing.md`: - Login type 5, a key byte, the token, the account and the password. - The login frame is zlib with no compression byte. Every later frame carries a compression byte: 2 below 0x30 bytes (4 if zlib is shorter), 4 below 0x1000, and 6 (bz2) from there. - XOR limits are 12 dwords for type 2 and 4 for types 4 and 6. These are caps on the group count. The count itself is one LCG step per 4-byte group present, so a body of 5 bytes gets 2 steps, and a partial last group still steps (`protocol/codec-and-framing.md`). - Listserver id 7 carries `GNW30123newmain`, with hosts from `arrays2.dat`.

2.31's send path is the same code as 2.22's: same size thresholds, XOR limits, and 0x40000 queue cap. **[bin]**

	2.22	**2.3**	**2.31**
Server → client ids handled	80	as 2.31 [inf]	81: drops 44 `HASNPCSERVER`, adds 84 `LARGEFILESIZE` and 189 `MOVE2` **[bin]**
Client → server ids sent	0–24, 26–30, 32–35, 37–41, 43, 44, 80	as 2.31 [inf]	2.22 minus 21 `PUTNPC`, plus 46 `ENTERLEVEL` (sent on every level entry including doors, links and server warps, lab **[cap]**; 2.22 sends no 46) **[bin]**
Highest PLPROP / NPCPROP parsed	74 / 73	as 2.31 [inf]	80 / 77 (X2, Y2, Z2) **[bin]**
Precise positions	no	yes **[srv][doc]**	yes **[srv][bin]**
After login	37, then 39 (10 bytes)	same	same **[bin]**

Notes on the table: - **Client → server set.** It was read from the call sites of the client's send helper. Id 44 (`PROCESSLIST`) was also seen in the 2.22 capture, sent as the reply to server id 182. Ids 36, 42 and 45 are not matched to a call site, so treat the set as a floor, not a complete list. **[bin][cap]** - **Tampercheck (39).** 10 raw bytes, identical at every login of a build and across accounts and login keys, so a per-build constant. **[cap]** 2.22 `250e4d0f962b03ede631` and 2.31 `291655b7602e87c9eb82` (2.17, same layout: `2a7bad195e231243d82c`). GServer-v2 reads and discards it, so the lab cannot say whether a different value would be rejected. The bytes are two chained CRC-32 words over the client's own memory image in a 5-byte non-GINT5 form (`client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`; words in `tampercheck-words.tsv`). The words were reproduced offline for 2.17.1, 2.22 and 2.31. Re-sends differ by build: 2.17 once at login; 2.22 at login, on every gmap edge crossing and after every client-initiated level-link request (not after a server-side warp), with no periodic send; **2.31 at login and then about every 3.2 s**. The 3.2 s cadence is a 50 ms timer counting modulo 64 (`client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md`). 2.31 additionally sends `PLI_SHOWIMGPLAYER` (24, a 1-byte payload) about every 5.1 s and `PLI_ENTERLEVEL` (46, 1 byte, value 1) on every level entry (lab, batch b2 and c; not once at login). 2.22 also sends `PLI_SHOWIMGPLAYER` (24) = 9 about every 5.1 s, but only while another player is in its level. The meaning of the id 24 payload is unknown. 2.31 also answers the server's process-list request with `PLI_PROCESSLIST` (44), as 2.22 does. **[cap]** - **2.22 login sequence in the lab.** The client sent 37 `English`, then 39, then 34 `UPDATEFILE` for its default images (sword, shield, body, bomb), 23 `WANTFILE`, and 2 `PLAYERPROPS`. It answered the server's 182 with 44. The `gr.x` / `gr.y` flag sets in the capture come from the server's movement NPC, not from the client. **[cap]** - **Zlib fix NPC.** GServer-v2 sends a `-gr_zlib_fix` weapon to every client from 2.21 to 2.31. It pads chat text made of long runs of one or two repeated characters. This is a server-side workaround for a client

compression bug, so a faithful 2.22–2.31 client should keep that bug or at least tolerate the padding. **[srv][inf] - 2.31 and `setmap`.** 2.31 does not request the map file after `setmap`, so GServer-v2 pushes every BIGMAP file at login for 2.31. **[srv]** It does not do this for 2.3, which suggests 2.3 still asked for the file. **[inf] - Precise movement.** From 2.3, GServer-v2 puts X2/Y2 before X/Y in other-player props and stops its `gr.x` / `gr.y` / `gr.z` movement-flag workaround. **[srv] - UDP.** Present in both builds, with the `noudp` and `fixedudpport` keys and the "Don't use UDP for data packages" option. `noudp` defaults to on. **[bin]**

What the player sees

Start window and server list

	2.22	**2.31**
Login and server-list backdrops	the "GraalOnline" banners (samurai house, then the character line-up)	**identical images [bin]**
Start-window buttons (seen live)	Training · Options · Get an account! · Start	Training · **Editor** · Options · Get an account! · Start. Editor launches `GraalEditor.exe`. **Training is still there in 2.31 [cap][doc]**
Server-list tabs	Gold · Silver · Bronze **[bin]**, possibly a Reborn edit	Graal Gold · **Graal 3D** · Classic · Hosted. A 3D server shows "You need to download Graal3D…" or starts `Graal3.exe` **[bin][doc]**
Tab for the same lab server (live)	**Bronze**	**Hosted**. Only tabs that hold a server are drawn. **[cap]**
Server-list buttons (live)	Show More · Donations · Connect	same **[cap]**
`graal://` and `graal3://` links	no	registered at start-up. A link sends `serverstartparams` to the server's Control-NPC **[bin][doc]**

For comparison, 2.17's tabs are Graal Gold · Classic · Hosted, and its start buttons are the same as 2.22's. **[bin]**

In-game windows and keys

- **The editor is gone from the 2.31 client.** 2.22 still carries the full level editor:
  - the toolbar (New, Open, Save, Cut, Copy, Paste, warp field, links, signs, flags, Export Tiles Array, Play)
  - the tile and object panels and the "Show blocking tiles / foreground / terrain grid / Hide npcs" checkboxes
  - the dialogs for chests, baddies, NPCs, signs, links, flags, characters, object names and the tile array
  - the F6 script debugger

  2.31 has none of these forms (18 forms instead of 29). **[bin]**
- **The F1 window, seen live.** 2.22 shows six buttons: General options, Edit level (F4), Load level (F5), Debugger (F6), Playerlist (F7) and Serverlist (F8). 2.31 shows four: General options, Load level (F5), Playerlist (F7) and Serverlist (F8). Edit level and Debugger are still in the form resource but hidden at run time, and pressing F4 in game does nothing. **[cap][bin]**
- **Options → General**: 2.22 has "Save downloaded images (to)". 2.31 replaces that checkbox with "Use filename cache (don't scan folders each startup)", config key `cachefilenames`, backed by `FILENAMECACHE.txt`. Every other tab is unchanged: Video, Audio (midi / mod-mp3 / radio / voice / sfx), Microphone (GSM / LPC / LPC-10 voice chat), Controls (joystick, DirectInput, Change keys), and the player-list options. **[bin]**
- **Key layout**: the Change Controls form is identical in the two builds. 2.31 fixes the saving and loading of the key configuration. **[bin][doc]**

- **HUD**: identical in a live session on the same server: A / S / D item slots, the rupee, bomb and arrow counters, the spinning heart icons and the MP bar. **[cap]** There is also no difference in the shipped images or the HUD image file names (`state.png`, `letters.png`, `sprites.png`, and so on). The two status icons `ss_arrow.png` and `ss_bomb.png` are no longer referenced in 2.31, and neither are `graalmap.png`, `headsconfig.txt`, `loadgmaps.txt`, `recordpos.txt` or `shield1.gif`. All of these belong to offline or editor play. **[bin][inf]**

Game behaviour

	2.22	2.3 / 2.31
Classic client-side mode (client applies NPC props itself when the server has no NPC server)	yes, honours `HASNPCSERVER`	removed **[bin][doc]**
Smooth positions	half-tile X/Y	full-precision X2/Y2/Z2 for players and NPCs **[bin][srv]**
NPCs crossing gmap level borders	buggy	fixed in 2.31 **[doc]**
Float-to-string in scripts	buggy	fixed in 2.31 (`#v` prints integers with more than 6 digits) **[doc]**
CPU use	about 99 % (busy timer)	new timer **[doc]**
Object lookup	linear	B-tree. Huge ganis are no longer culled early **[doc]**
Local editor flags (`local.`, `this.-`, `gotheart_`)	present	gone with the editor **[bin]**

GS1 script differences (from the client's symbol table) [bin]

Change	**2.17 → 2.22**	**2.22 → 2.31**
Added	`gravity`; `textwidth(zoom,font,style,text)`; `textheight(zoom,font,style)`; `isthisflagset` and `isplayerflagset` (replacing `isflagset`); one new # message code with a string argument (`#R`)	`timevar2`; `downloadsize`; `onwall2(x,y,w,h)`; `onwater2(x,y,w,h)`
Removed	`isflagset`	none (the command list is unchanged)

`downloadsize` is announced in the 2.22 notes, but it is only in 2.31's symbol table, which fits the 2.3 note "is correctly giving the size". A 2.22 profile should treat it as undefined (0). **[bin][doc][inf]**

Using this in Seihai

The `features` block in the TOML gives one switch per difference above. The three that change the most for the player:

1. `editor_in_client` (2.22 only): F4 and F6, and the Edit level and Debugger buttons in F1. Training and Load level (F5) stay in both.

2. `serverlist_tabs`, `graal3d_tab` and `url_protocols` (start window).
3. `precise_move` (2.3+), with the matching PLPROP 78–80 and NPCPROP 77 parsing.

Screenshots from the 2026-10-30 live run (beta4 lab server) are in the project's `graal-decompile/live-trace/shots/v222-231/`.

Open

- A 2.3 binary, to confirm that 2.3 matches 2.31 apart from its bug fixes.
- Call sites for client → server ids 36, 42 and 45 (the 2026-10-24 decompile pass covered the receive handlers and the tamper builder, not these send sites).
- Whether the 2.22 tab names Gold, Silver and Bronze are original or a Graal Reborn edit.
- The meaning of the 2.22 `#R(string)` message code.

`client-internals/live-checks-editor-offline-sound.md`

Live checks: editor, offline mode and sound (2.05, 2.22, 2.31, GraalEditor)

Date: 2026-10-30

These results come from running the real clients in the Wine lab described in `live-trace-205.md`: Wine 9 with a 32-bit prefix, Xvfb, a network namespace with only loopback, and an ALSA `null` device so that DirectSound starts. File access was logged with Wine's `+file` channel, and which file each action used was read from that log. The code paths named below come from the 2.05 rev 2 decompile. Names only, no code.

Tags: **[live]** seen in a running client · **[2.05]** read from the 2.05 binary · **[inf]** inferred.

Corrections to earlier notes are collected at the end.

Offline mode

The Training button

All three clients tested have a **Training** button on the start screen. 2.05 rev 2's welcome text says: "press 'Training' for some offline training levels, and 'Start' for the online mode." Pressing it starts offline play on the embedded tutorial and makes no network connection. **[live]**

Client	Start screen buttons	Training starts on	Big map / minimap
2.05 rev 2	Training, Options, Start	`tutorial_b-4.graal`: the welcome sign, Stefan, the two coloured lights	`tutorialmap.txt` set by the level's `setmap` NPC
2.22	Training, Options, Get an account!, Start	`offlinetutorial_b-2.graal`, the centre of the 3×3 grid: wizard, cauldron, two torches	the client's `bigmap.txt` / `minimap.txt`, plus `maps/parts/offlinetutorial/offlinetutorial_*.graal.png`
2.31	Training, **Editor**, Options, Get an account!, Start	`offlinetutorial_b-2.graal`, the same as 2.22	the same as 2.22

The 2.22 and 2.31 start level plays `cauldron_bubbles.wav` and `fire_torch0..2.wav` as soon as it loads. Those sounds come from its NPCs. **[live]**

The embedded level always wins

A file on disk with the same name does not replace the embedded level. **[live]**

- **2.05**: with `tutorial_b-4.graal` (a different level's contents) in the game folder, Training still loads the embedded b-4. The client does not even look for the name on disk.
- **2.22**: with `offlinetutorial_b-2.graal` in the game folder, the client looks the name up once but still loads the embedded copy.

Tile definitions when offline

Every client tested reads `levels/tiledefs/tiledefsOffline.txt` when Training starts. That includes 2.05, which builds the file name as `tiledefs` + the server name + `.txt`, and the server name offline is `Offline`. The shipped file has one line, `picso.png,offlinetutorial,1,0,0`, which applies to the `offlinetutorial*` levels only. So in 2.05 the read happens but has no effect on the old `tutorial_*` levels. **[live][2.05]**

The reconnect dialog and *Play offline*

Pressing *Start* in 2.05 with no listserver shows "The connection couldn't be established!" and stays on the server list. That screen has no *Play offline* button. **[live]**

The reconnect dialog has three buttons: *Reconnect*, *Open serverlist* and *Play offline*. It was reached two ways in the lab. **[live]**

How the connection ended	Dialog text	*Play offline* goes to
The listserver listed a server, but its game port refused the connection (TCP reset)	"The connection couldn't be established!"	`tutorial_b-4.graal` with the welcome sign, the same as Training. No level file was read from disk.
Logged in, then the game socket was destroyed from outside (`ss -K`)	"You have been disconnected from the game server."	The same level, reloaded from the game folder (`onlinestartlocal.nw`), with the player where they stood.

Two lab notes:

- Closing the server side normally (a FIN) was not enough. The client left the socket half-closed and showed no dialog. The dialog came up only after a reset or a socket error.
- For the second row, a copy of the online level was in the game folder. The case where the level is not on disk was not repeated after a drop. The code says it falls back to the tutorial (see `offline-tutorial-205.md`).

Built-in editor (2.05)

Generate map (M)

After the "Generating the map may take some minutes. Proceed?" prompt, the editor works through the level list of the **currently loaded big map**. That is the list from `bigmap.txt`, or whichever list a `setmap` NPC loaded (for the tutorial, `tutorialmap.txt`). **[live][2.05]**

- It reads each listed level from disk in list order, row by row, and loads it as a level. Then it runs the level's NPCs into a 1024×1024 view and shrinks that to **56×56 px** in the output image. Missing or unreadable entries become blank squares.
- It does not follow links. Levels that are not in the list are skipped, for example `tutorial_b-2_house.graal`.

- The result is offered in a save dialog as `graalmap.png`, defaulting to the level's folder. With the 3×4 tutorial list it produced a 168×224 PNG.
- Afterwards, the level that was open in the editor is restored.

Play saves first

Pressing *Play* on a named, modified level writes the level to disk ("Level saved" in the status bar) before the *Level options* dialog asks for the start position. **[live]** This matches the 2.22 behaviour in `tools/level-editor/222-reference.md`.

Edit flags

Edit flags edits the client's in-memory flag list. Flags typed into it are still there after Play and a return to the editor. Nothing is written when the dialog closes or when the client exits: the test flag was not in any file in the game folder or the Wine prefix, nor in the registry. The list does not survive a restart. **[live]**

2.31 and GraalEditor

- The 2.31 start screen's **Editor** button starts `GraalEditor.exe` from the game folder as a separate process. The 2.31 client still has no editor forms of its own. **[live]**
- GraalEditor 2.3.1.1 runs under Wine. It opens on a new `new2.nw` tab with the four palette modes, including *Predefined NPCs*. **[live]**
- At startup it looks for `arrays2.sig`, `arrays2.dat`, `graaleditor.sig`, and `arrays.dat`. The shipped folder has no `GraalEditor.sig`, and the editor starts normally anyway. With one byte of `arrays2.sig` flipped, it also starts normally, with the same file reads and no message. Whatever the RSA check controls, it does not stop the editor from starting. **[live]**

Sound (2.05)

With an audio device present, DirectSound starts and the client plays effects. Each effect shows up as a lookup of its file under `sounds\`, and that is how the rows below were matched to actions. Most rows were triggered in a test level: water, a pushable stone block, a bush, ledge rows, and chests. GS1 NPCs fired the remaining effects on staggered timers (`putbomb`, `putexplosion2`, `shoot*`, `putleaps`, `spyfire`, `lay`, `puthorse`, `putcomp`, `hurt`, `setgender`). **[live]** marks a sound heard in the running client. **[2.05]** marks a sound known only from the call sites of the sound routine.

Action	Sound	Source
Walking on ground	`steps2.wav` about every 0.2 s; `steps.wav` when a move starts	[live]
Swimming	`water.wav` about every 0.8 s	[live]
Riding a horse	`horse.wav` and `horse2.wav`, alternating about every 0.4 s	[live]
Sword swing	`sword.wav`; with the battle axe, `axe.wav`	[live]
Lifting a bush	`lift.wav` for a male player, `lift2.wav` for a female one (`setgender female`)	[live]
Getting hurt	the same `lift.wav` / `lift2.wav`: `hurt`, an explosion, a thrown ball or a baddy hit all play it	[live]
Dying	`dead.wav`	[live]
Laying a bomb	`put.wav`	[live]

Action	Sound	Source
Bomb or explosion	`bomb.wav` (`putbomb` of any power, `putexplosion`, `putexplosion2` power 2, the end of a fireblast or nuke)	[live]
Jolt explosion	`thunder.wav` (`putexplosion2` power 3)	[live]
Launching any projectile	`arrow.wav` for arrows, fireballs, fireblasts, nukes and `shootball`, and for arrows from shooting baddies and the dragon	[live]
Fire spy	`fire.wav` (`spyfire`)	[live]
A thrown object or leap landing	`crush.wav`; `water.wav` for the water leap (type 5)	[live]
Picking up a laid item (rupees, battle axe)	`extra.wav`	[live]
Opening a chest	`chest.wav`, also when the chest has a sign. No item sound follows.	[live]
Sign pages	`nextpage.wav` on every page turn	[live]
Pushing a stone block	`stonemove.wav` about once a second while it moves	[live]
Ledge jump	`jump.wav`	[live]
Frog	`frog.wav` when it hops	[live]
Baddy walking	`steps.wav`	[live]
Baddy dies	`compudead.wav`	[live]
Baddy draws its sword	`swordon.wav`. Six baddy types were tried; none played it.	[2.05]
An arrow hitting the player	`arrowon.wav`. Not reached: no test arrow hit the player.	[2.05]
`item.wav`, `item2.wav`	Only used in the sign-reading routine and in a server-message handler. The chest and laid-item tests didn't play them.	[2.05]
Another player's bomb or item (online)	`bomb.wav` / `thunder.wav`, `item.wav`, `extra.wav`, from server messages	[2.05][inf]
GANI `PLAYSOUND`, GS1 `play` / `playlooped`	the named file	[2.05]

The lift/hurt sound is picked by a single byte on the player object: `lift.wav` when it is set, `lift2.wav` when clear. New players start with it set. That `setgender female` clears it is inferred from the sound changing. **[2.05] [inf]**

Corrections to earlier notes

- `offline-tutorial-205.md` said there is no offline item on the start screen. There is: **Training**. It is the usual way in, and *Play offline* on the reconnect dialog is the other.
- `offline-tutorial-205.md` and `tools/graaleditor-vs-205.md` said only 2.22 reads `tiledefsOffline.txt`. 2.05 and 2.31 read it too (see above).
- `tools/graaleditor-vs-205.md` said 2.31 has no editor. It has no editor *forms*, but its start screen launches GraalEditor.
- `level-editor-205.md` listed Generate map and Edit flags as open. Both are answered above.
- The first version of the sound table in this note guessed that `lift2.wav` is for heavy objects and that `fire.wav` is the fireball. Both were wrong. `lift2.wav` is the female lift and hurt sound, every projectile launches with `arrow.wav`, and `fire.wav` belongs to `spyfire`.

`client-internals/player-mannerisms-live.md`

Player mannerisms seen in real clients (1.39, 1.41, 2.05 rev 2, 2.22)

Date: 2026-10-30. What the player does, tick by tick, in four real clients run under Wine with no network.

- **2.05 rev 2.** The player state was read every 3 ms from outside the process while xdotool held keys, and the screen was recorded at 60 fps. Tag **[live]**. The test level is `lab1.nw` (described below).
- **1.39, NewMain 1.41 (`graalnm24`) and 2.22.** These were traced the same way, from their own player records (see "Other builds" below). Tags **[1.39]**, **[1.41]**, **[2.22]**. Where only video was used, the tag is **[video]**, and those counts are good to ±1 tick.
- **The gani files.** These are the files that the timings come from. Tag **[gani]**.

A tick is the 50 ms game tick. Speeds are in tiles. This note has no addresses and no decompiler text. The harness, the level, the raw traces and the gani comparison script are in the project files under `graal-decompile/live-trace/mannerisms/`. Stills are in `mannerisms-shots/`.

Summary table (2.05 rev 2)

action	what the client does	timing
walk	`walk`, 0.5 tile per tick, one gani frame per tick (8 frames)	8-tick cycle
key released	`idle` in the same tick, frame 0	0 ticks
run into a wall	`walk` keeps cycling in place; `push` on the **18th** blocked tick	0.9 s
push pose	5 frames × 4 ticks; restarts at frame 0 each time the 20-step push cycle wraps	20-tick loop
slide along a wall	holding Down into a wall and adding Right walks right at full speed with `walk`. The blocked count drops to 1 and stays there.	—
lift (A)	`lift` for **6 ticks**, then `carrystill`. The object counts as carried from the 5th tick.	0.3 s
carry walk	`carry`, still 0.5 tile per tick, 8 frames × 1 tick	—
hide (D while carrying)	`hiddenstill` / `hidden`; walking is **0.1 tile per tick**; only the object is drawn	—
throw (A, or S from hiding)	`idle` in the same tick	0 ticks
sword (S)	`sword` 5 frames × 1 tick, then `idle`. The player stops while it plays.	5 ticks
spin (S again during a swing, has-spin status)	the swing is cut off in the next tick and `spin` plays: 13 frames × 1 tick, then `idle`	13 ticks (0.65 s)
hurt	`hurt` for 31 ticks (12 if an arrow is held); knockback 1 tile per tick for 5 ticks; blinking and no further hits for 40 ticks	1.55 s / 2.0 s
swim	`swim` from the first tick the point (x + 1.5, y + 2) is on water; 0.5 tile per tick; 8 frames × 2 ticks; **keeps animating when standing still**	16-tick cycle
grab / pull	`grab` on A at a wall. `pull` starts on the arrow away from the wall: 3 frames × 2 ticks, with a pull try every 18 ticks.	—

Walking, idle and running in place [live]

- Each walk tick moves 0.5 tiles and advances the `walk` frame by one (0 to 7, then 0). The first tick after the key goes down already moves the player.
- Letting go of the arrow switches to `idle` frame 0 in the next tick. There is no slow-down and no finishing step. In `idle` nothing in the player state changes from tick to tick.
- **Against a wall.** A refused step still advances the walk frame, so the player runs in place. The blocked-step count goes 1, 2, …, 17, and `push` replaces `walk` on the 18th. This is the same count as the ledge jump (`ledges-push-pull.md`). Deep water (tile 3644) counts as a wall.
- **The push pose.** Each push frame lasts 4 ticks (`push.gani`: 5 frames, each `WAIT 3`). When the count reaches 38 it goes back to 18, and the gani starts again from frame 0. Five frames of 4 ticks make exactly that 20-tick cycle, so the pose loops seamlessly.
- **Sliding along a wall.** Still holding Down into the wall, add Right. The player walks right at the full 0.5 tile per tick in `walk`, and the push pose is gone in that tick. Let go of Right and the player runs in place again from walk frame 0. The blocked count goes on from where it had dropped to (1), so the full 18 ticks have to pass again before `push`.
- Pressing the arrow again into the wall after letting go starts the count at 1, with no push straight away.

Lifting, carrying, hiding and throwing [live]

Test objects: a bush, a pot, a light stone, a black stone and a sign in a row (`lab1.nw`, y = 20).

- **Lift.** A at the bush (or the pot) plays `lift` at once. A five-step counter runs 4 down to 0, and the carried flag is set on the 5th tick. `carrystill` follows on the **7th tick** (6 ticks of `lift`, as `lift.gani` = 1 frame with `WAIT 5`). The bush tile turns into its stump at once (`arrays.dat` array 12).
- **Carry.** Walking while carrying plays `carry` (8 frames, one per tick) at the normal 0.5 tile per tick. There is no slow-down. Stopping plays `carrystill`. The object is drawn over the head. In the video the head and arms are hidden behind it (`205-pot-throw.png`, first frame).
- **Hide** (D while carrying a bush). In the same tick the pose becomes `hiddenstill`, and the player's "hidden" flag (STATUS bit 0x02, see `carry-hide-and-glove.md`) is set. Only the bush is drawn, where the player stands. Body, head and shadow are all gone (`205-hidden-under-bush.png`).
  - Walking while hidden plays `hidden` at **0.1 tile per tick**. That is a fifth of the walk speed, which confirms the 0.2 factor in `carry-hide-and-glove.md`. The gani frame does not advance: `hidden.gani` is one frame with `WAIT 4` and a step sound.
  - Stopping goes back to `hiddenstill`.
  - D again leaves the hiding place, back to `carrystill` in the same tick.
  - S while hidden throws the bush. The player is in `idle` in that tick, with the hidden flag cleared and the carried flag cleared.
- **Throw** (A while carrying). The player is in `idle` in the same tick. Video of a pot thrown upwards (`205-pot-throw.png`, one frame per tick):
  - The object flies about 6 tiles in 9 ticks. Its shadow moves along the ground while the object itself is drawn rising and then falling back onto the shadow.
  - It breaks into four pieces that spread out over 4 ticks and then vanish.
  - The pot's base tile (the grey plinth) stays where it was lifted.
- A at a black stone with glove 1 does nothing: no lift and no grab. The stone is not a wall tile.

Sword [live]

- S plays `sword` in the next tick. Five frames of one tick each, then `idle` (`sword.gani`: 5 frames, `SETBACKTO idle`).
  - The player does not move while it plays, even with an arrow held.
  - With the arrow still held, walking resumes in the tick after the swing, from walk frame 0.
  - Holding S does not swing again. Every swing needs a new press.
  - Holding S for 2.5 s gives no charge and no spin, with or without the spin status. See the next section.
- **What it looks like** (facing up, `205-sword-up-bush-cut.png`):
  - The blade sweeps from the right side over the head to the left.
  - The bush in front is cut on the **first** swing frame. The leaves fly up and out for about 8 ticks, and the stump is left behind.
  - The shield stays drawn on the right during the swing.
- **Offline editor play with no sword.** A player started from the editor has sword power 0. S still plays the full `sword` gani and sound, but no blade is drawn and nothing is cut. Give the player a sword (`setsword sword1.gif,1` in a level NPC) to test cutting.

Spin attack [live, online]

Recorded on the online lab: the stub server sent `STATUS` (prop 18) = 0x54 (male, weapons allowed, has spin). The offline editor has no way to set the has-spin status.

- **It is a second press, not a charge.** In 2.05 the spin starts when S is pressed again while `sword` is still playing. Holding S down does nothing extra.
  - The decompile agrees: the S handler checks "already in the sword mode (6) and has spin", then switches to the spin mode (13) instead of starting a new swing.
  - The client only sees a new press if S was up for at least one tick in between. A release and re-press inside the same 50 ms tick is missed.
  - A press on swing frames 2, 3 or 4 all worked. A press after the swing has ended (the player is back on `idle`) starts a normal swing, not a spin.
  - A press during the spin is ignored. It does not queue another spin or a swing.
- **Timing.**
  - The swing is cut off: `spin` frame 0 shows in the tick after the second press, so a press on frame 3 skips sword frame 4.
  - `spin` is 13 frames of one tick each (650 ms), then `idle` in the next tick.
  - The client stores the facing direction when the spin starts.
  - The client sends `GANI=spin` to the server once at the start and `GANI=idle` at the end, and asks for `spin.gani` the first time.
- **What it looks like** (`205-spin-attack.png`, facing down; first row is the swing, then the spin):
  - The blade sweeps once round the player with a pale blue trail, starting from the facing direction and turning down, right, up, left and back to down.
  - The body turns with the blade, so the back view shows when the blade is above.
- **2.22 does the same** [2.22]. It was run online against the patched GServer-v2 beta4 lab server, with `STATUS 84` in the account file.
  - A second press on swing frame 3 cut the swing off, and `spin` ran 13 × 1 tick, then `idle`.
  - Holding S gave one swing only, and a third press during the spin was ignored.
  - The 2.22 pack's own `spin.gani` looks the same, blade trail included (`222-spin-attack.png`).
  - A brand-new account created with `STATUS 84` once stalled on "Loading account…"; an existing account edited to 84 logged in fine. Not chased further.

- Not checked: which bushes the spin cuts and whether it hits players or NPCs.

Hurt, knockback and blinking [live]

The test NPC runs `if (playertouchsme) { hurt 1; }`.

- Touching it plays `hurt` in the same tick and takes half a heart (3 → 2.5).
- **Knockback.** The player moves **1 tile per tick for 5 ticks**, along the direction from the NPC's image centre to the player's centre. In the run it went from (29.5, 48) to (28.79, 52.95), which is 5.0 tiles. This is exactly the decompile's description (`decomp-findings-2026-10-25-k/01-knockback-and-packet-hits.md`).
    - If the player is inside a blocking NPC's image, every knock step is refused and the player does not move. A large image, such as `bomb1.png` at 128 × 320, makes this happen.
- **Pose.** `hurt` is shown for **31 ticks** (`hurt.gani`: 1 frame, `WAIT 30`), then `idle`. The hurt pose keeps the facing.
- **Can't be hurt again** for 40 ticks (2 s) from the hit. The second touch in the run came only after that.
- **Blinking.** For the whole 40 ticks the player's body is drawn in one tick and left out in the next: only the shadow shows on the "off" ticks (`205-hurt-knockback-blink.png`). It goes on after `hurt` has given way to `idle`.
- **Walking again with an arrow held.** The hurt counter starts at 40 and drops by one each tick. While it is above 29 the arrows are ignored. From the 12th tick after the hit (counter 28), a held arrow walks again and `walk` replaces `hurt`. So `hurt` lasts 31 ticks only when no arrow is held. Re-run 2.05 with Left held through the hit: hurt at tick 88, knockback ended at tick 93, walking at tick 100. Blinking and the 40-tick no-rehit go on regardless.

Swimming [live]

Water tile 461 (passable, liquid); the pool starts at x = 40.

- The switch to `swim` happens on the tick where x + 1.5 reaches 40 (x = 38.5). Leaving the water happens where x + 1.5 drops below 40 (x = 37.5). In that tick `walk` comes back, at frame 0.
- Swimming speed is the same 0.5 tile per tick as walking.
- `swim` steps one frame every **2 ticks** (8 frames, `WAIT 1`), so the stroke takes 0.8 s.
- **It keeps animating while the player is not moving.** It does not reset when the player turns.
- The splash at entry and exit is in `water-and-ganis.md`.

Grab and pull [live]

At a wall of deep-water tiles, facing down:

- A at the wall plays `grab` at once and holds it.
- Holding the arrow away from the wall (Up) plays `pull`: 3 frames of 2 ticks each (`pull.gani`, `WAIT 1`). The pull cycle counter goes 2 … 19 and back to 2, so it repeats every 18 ticks, as in `ledges-push-pull.md`. Nothing moves against a wall.
- **Letting go of the arrow while still holding A.** The cycle counter drops to 1, but the pose stays `pull` and keeps animating. It does not go back to `grab`.
- **Letting go of A as well.** The pose still stays `pull`, animating in place, until the next key moves the player. This answers the open question in `ledges-push-pull.md`: nothing sets `idle` there, and the stale pull loop is what is drawn.

Other builds: 1.39, NewMain 1.41 and 2.22

How each was run

- **1.39** is the last client before ganis, and its player animation is built into the executable. It runs in its own One Player mode, which loads `levels/start1.graal`.
  - The test level was rewritten as a `GR-V1.03` file with the same objects (`mkgraal.py`). The objects are moved so that the view does not clamp at the edges: the row of five at y = 24, the water at x 42 to 59, the wall at y = 44, and the bushes at y = 34.
  - The player starts at (21.5, 28).
  - Keys reach the game only with `nodirectinput=true` and `joystickon=false` in `graalconfig.txt`.
- **NewMain 1.41.1** (`graalnm24`, January 2001) is the first gani client. It is an update pack: copy it over the 1.39 folder, because it needs 1.39's `headsconfig.txt` and GIF heads. It opens a `.nw` in the editor and plays it the same way as 2.05 (`startnw.sh`).
- **2.22** (`Graal_222.exe` in the Reborn folder) also opens in the editor. Its Load dialog did not take the typed start position, so the player started at the default (30.5, 30).
- **Finding the player record.** In each build the record starts with its class pointer, followed by x at +4 and y at +0xC (doubles), as in 2.05. One global points to it. `diffscan.py` finds it by walking a known distance; it checks doubles at both 4-byte alignments.

Cross-version table

"=" means the same as 2.05.

behaviour	**1.39**	**1.41**	**2.05 rev 2**	**2.22**
walk speed / frame cycle	0.5 per tick, 8 ticks [video]	=	0.5 per tick, 8 frames × 1 tick	=
blocked count: push at 18, wrap 38 → 18	= (the same counter)	=	yes	= (push in the tick after 18 blocked ticks)
push pose frames	about 2 ticks each [video]	4 ticks (`push.gani`)	4 ticks	= (5 × 4 ticks, looping)
lift	counter 4 → 0, carried on the 5th tick; no lift pose of its own	`lift` 6 ticks, then `carrystill`	=	=
carried-object id	261 for a bush (the same sprite number)	–	261	–
hide (D while carrying)	hidden flag; walk 0.1 per tick	hidden flag, but the pose **stays `carrystill` / `carry`**, with the legs cycling at 0.1 per tick	`hiddenstill` / `hidden`	= 2.05
S throws from hiding	yes	yes	yes	yes
S throws while carrying (not hidden)	–	–	yes [dec] (not traced live)	**yes**: `carrystill` to `idle` in the tick after the press, no `sword` frame
sword	counter 1 → 6 (5 ticks); cuts the bush	`sword` 5 ticks; cuts the bush on the first frame (`141-sword-up-bush-cut.png`)	=	–
hurt: lock-out / no-rehit	40 ticks	40	40	=
knockback	**4 steps** of 1 tile (4 tiles)	5 steps (5 tiles)	5 steps	= (4.9 tiles on the same test)
hurt pose	about 10 ticks, then the normal pose [video]	`hurt` 31 ticks	31 ticks; 12 with an arrow held	= (12 with an arrow held)

behaviour	1.39	1.41	2.05 rev 2	2.22
blinking	starts **only after the hurt pose**, then every other tick until tick 40 [video]	from the hit, every other tick	from the hit	–
swim entry / exit point	x + 1.5 on liquid	=	=	=
swim speed	0.5 per tick	=	=	=
standing still in water	the water ring keeps alternating; the head stays still [video]	**`swim` frame resets to 0 and freezes**	keeps animating	keeps animating; the frame resets to 0 at the stop, then runs on
letting go of the arrow while pulling	–	back to **`grab`**	stays `pull`, looping	stays `pull`, looping
letting go of A after a pull	–	`grab` stays until the next action	`pull` loops until the next move	= 2.05

Notes per build

- **1.39 hurt** [1.39]. The touch NPC uses the same `hurt 1`. The pose shows the player's back with the arms out (`139-hurt-then-blink.png`).
- **1.39 sliding past a blocking NPC** [1.39]. Walking right into a character NPC's side nudged the player along it by 0.125 tiles per tick. The player got around it, and `playertouchsme` did not fire on that contact. A timer NPC that tests `playerx` / `playery` was used for the hurt test instead.
- **1.39 swim** [1.39]. Only the head and a water ring are drawn. Standing still, the ring alternates between two shapes (`139-swim-standing.png`).
- **1.41 hide** [1.41]. The `graalnm24` gani folder does hold `hidden.gani` and `hiddenstill.gani`, but the 1.41 client does not play them. The hidden player keeps the carry poses, and the leg cycle still runs one frame per tick at a fifth of the speed.
- **1.41 sword** [1.41]. With `setsword sword1.gif,1` the blade is drawn and the bush in front is cut on the first swing frame, as in 2.05.
- **2.22 sword while carrying** [2.22]. In the `t222` run the bush is lifted, D hides, the player walks hidden, D comes out (`carrystill`), then S. The tick after S shows `idle`, with no `sword` frame: the sword key throws, as the 2.05 key handler does. 2.31 was not run. Not run in any build: S during the six `lift` ticks, and a throw with an arrow held.
- **2.22** [2.22]. Lift, hide, the stuck pull loop, swimming in place, push and hurt all behave as in 2.05. The hurt test was the same walk into the touch NPC from the right, with the arrow held, and gave the same knockback and the same 12 ticks before walking resumed. The only difference seen is that `swim` restarts at frame 0 when the player stops.

5.x and 6.x Windows clients

`protocol/v5v6-lab-play.md` has the same tests on 5.24r4, 5.30r3 and 6.03r7, timed from their own messages on a lab server. Walk 0.5 tile per tick, `push` at about 0.9 s, `lift` for 0.3 s then `carrystill`, throw in the same tick, and `sword` for 5 ticks with the bush cut as the swing starts are all the same as 2.05. New in those builds: while pushing, the client sends the trigger action `pushed,0` once a second. Chat is outlined text with no bubble, and it is cleared by the client after about 3 s. Hiding under a bush or stone (status bit 0x02, only the object drawn, 1.5 px per tick) and the pot's "D does nothing" also match 2.05. Grab and pull start in the next tick, and the client sends `pulled,1` 0.85 s into a pull. A hit plays `hurt` for 1.55 s, which is the 31 ticks of the `hurt` gani below, and pushes the player 5 tiles in 5 ticks. Swimming is as fast as walking.

The built-in ganis across the gani sets [gani]

The 2.0x archives in the project hold only the executables, and the 2.05 client fetches its ganis from the server. The gani sets on hand are:

- NewMain 2.4 and 2.6, the first gani clients (December 2000 files);
- NewWorld beta 1 (August 2001);
- the Graal Reborn 2.22 pack.

NewMain 2.4 and 2.6 are identical. In the table below, frames × ticks gives the ticks per frame (1 + WAIT); ">x" is SETBACKTO x; L means LOOP.

gani	NewMain 2.4 / 2.6	NW beta 1	2.22 pack
idle	1	1	1
walk	8 × 1, L	8 × 1, L	8 × 1, L
walkslow	8 × 2, L	–	8 × 2, L
sword	5 × 1, >idle	5 × 1	5 × 1, >idle
spin	13 × 1, >idle	–	13 × 1, >idle
push	5 × 4, L	**3 × 4**, L	5 × 4, L
pull	3 × 2, L	3 × 2, L	3 × 2, L
grab	1	1	1
lift	1 × 6, >carrystill	**6 × 1**	1 × 6, >carrystill
carry	8 × 1, L	8 × 1, L	8 × 1, L
hidden	1 × 5, L, single direction	(no frames)	1 × 5, L, single direction
kick	1 × 5, >idle	1 × 5	**1 × 11**, >idle
shoot	1 × 11, >idle	1 × 11	1 × 11, >idle
hurt	1 × 31, >idle	31 frames (10 × 2 then 21 × 1)	1 × 31, >idle
dead	**13 × 4** (52 ticks)	5 frames, >gameover	**15 frames** (3 × 7 + 2 × 8 = 37 ticks)
swim / lava	8 × 2, L	8 frames (6 × 2 + 2 × 1)	8 × 2, L
ride	4 × 2, L	4 × 2, L	4 × 2, L
sleep	1	**2 × 19**, L	1

- Every 2.05 live timing above matches the NewMain / 2.22 column: walk, push, pull, lift, carry, sword, hurt and swim.
- **The hidden ganis.** In NewMain, `hidden` and `hiddenstill` draw only a shadow, and the client itself draws the carried object. The 2.22 pack adds the object to the gani as an `ATTR3` sprite (`bushitem.png`). A 2.22 server's ganis drawn by the 2.05 client would therefore show the bush twice.
- Every set keeps the same sounds on the same frames: `steps2.wav` on walk frames 0 and 4, `sword.wav` on frame 0, `lift.wav` on lift and hurt, and `water.wav` / `water2.wav` on swim / lava.
- The full per-file output, with frame-content hashes, is `gani-compare.txt` in the project folder.

Asset gotcha: 2.05 loads only GIF heads, bodies and swords [live]

Given the 2.22 folder, where heads, bodies, swords and shields are PNG, 2.05 rev 2 looks only for `head0.gif` and the like. It finds nothing and draws the player with no body. That is the "missing body" in `live-trace-205.md`, and it is not a Wine bug. Converting `levels/{heads,bodies,swords,shields}/*.png` to `.gif` next to them fixes it. A Seihai 2.05 profile that resolves player images should try `.gif` only.

What Seihai does differently (from its `client/src/pose.rs`, 2026-10-30)

Seihai was not changed here. These are what a 2.05 profile would need:

1. **Push.** Seihai shows `push` after 17 blocked steps and advances it one frame per blocked step. The real client shows it on the 18th and holds each frame for 4 ticks (it honours `WAIT`).
2. **Pull.** Seihai steps `pull` once per tick; the real client steps it once every 2 ticks. Seihai drops the pose on release; the real client leaves `pull` looping.
3. **Gani WAIT.** Seihai does not read per-frame `WAIT`. The real hurt pose is 31 ticks with no arrow held, or 12 ticks before a held arrow walks again (Seihai uses a 500 ms minimum). Lift is 6 ticks. Swim is 2 ticks per frame, which Seihai already uses.
4. **Swim while still.** Seihai steps `swim` per step taken. The real client keeps it animating when the player stands still in water.
5. **Hurt blink.** There is no blink in Seihai. The real client draws the body on alternate ticks for 40 ticks.
6. **Hide.** There is no `hiddenstill` / `hidden`, no 0.1 tile per tick walk and no hidden flag in Seihai.
7. **Lift pose.** Seihai goes straight to `carrystill`. The real client plays `lift` for 6 ticks first.
8. **Spin.** Seihai records the `spinattack` flag but never plays `spin`. The real client turns a second S press during a swing into a 13-tick spin.

Harness

- `act.py <name> "<keys>"` runs a key script, for example `down:Up w:0.6 up:Up tap:a`. It records the player state (gani, frame, position, every changed field) and 60 fps video at the same time. A `tap` holds the key for 120 ms: shorter taps are lost between ticks. `NOMEM=1` records video only, for builds whose player record has not been mapped.
- `sh.py` prints a trace, with the key presses interleaved.
- `mkgraal.py` writes the 1.39 test level as `GR-V1.03`, with NPCs given on the command line. `start139.sh` starts 1.39 in One Player mode, and `startnw.sh <dir> <exe> <level> <x> <y>` starts an editor-era build on a `.nw`.
- The per-build settings for `act.py` (process name, player-record global, gani offset) are in `README.md` in the project folder.
- `act.py` also takes `gx:` / `gy:` steps, which walk until the player reaches that x or y.
- `mklevel.py` writes `lab1.nw`:
  - five liftable objects in a row at y = 20;
  - a pool of water tile 461 at x 40 to 55, y 10 to 30;
  - a deep-water wall at y = 40;
  - four bushes at y = 50;
  - a `setsword` NPC, a touch-to-hurt NPC at (30, 44) and a `lay spinattack` NPC.
- `gani.py` summarises and compares gani folders.

Open

- **The spin attack in 1.39 and 1.41.** It was seen only in 2.05 and 2.22, online. Offline, neither `lay spinattack`, `set canspin` nor a `spinattack` chest gave the has-spin status.
- **1.39's push and walk frames** are not in its player record as plain numbers; they were read from video only.

`version-profiles/matrix/builds-deltas.md`

Per-build deltas from the static survey, 1998 to 2.31

Date: 2026-10-30. Generated from the survey in `README.md` (method there). Each line lists what a build adds (+) or drops (−) against the build listed before it, in link order. Graal3D is a separate program and is listed on its own. **[bin]** strings and form resources; **[file]** shipped files.

`graalconfig.txt` keys

Plain `key=` strings. From 1.38 some keys are built one character at a time, so a key can be missing from this list while the build still reads it; garbled three-letter keys (`ekv`, `u5j`, `d4n`, `kxs`, `azg`) are such fragments.

- **6th Graal (beta5)**: + `joystickon=`, `musicon=`, `reversestereo=`, `soundfxon=`, `waitvertical=`
- **8th Graal (beta8)**: + `saveimages=`
- **1.1 (15th)**: + `editnickname=`, `gamemode=`
- **1.21**: + `accountname=`, `loadcolorsfromserver=`, `password=`, `serveraddr=`, `serverport=`
- **1.28 beta2**: + `compabilitydraw=`, `externalchat=`, `language=`, `nodecompressionerror=`
- **1.3 test**: + `font=`, `fontsize=`; − `accountname=`, `password=`
- **1.31**: + `fixedport=`, `fixedudpport=`, `noudp=`; − `editnickname=`
- **1.32**: + `nomassmessages=`
- **Graal3D** (separate program): `bodytex=`, `bodyx=`, `fixedport=`, `fixedudpport=`, `headx=`, `noudp=`, `serveraddr=`, `serverport=`
- **1.33**: + `nodirectinput=`
- **1.35**: + `nohtmlinpms=`, `u5j=`
- **1.36**: + `limitnicknames=`, `nicknamelimit=`; − `serverport=`
- **1.38**: − `externalchat=`
- **1.39**: + `dontconnectlevels=`, `dontloadlistheads=`, `nohtmlimages=`, `notoalls=`, `sortplayersbyname=`
- **1.41 rev1**: + `dontsavepasswords=`, `fullscreenmode=`, `usewinamp=`; − `serveraddr=`
- **NewMain 1.42 (24)**: + `dontsavepms=`, `dontusedirectx7=`, `ekv=`, `head=`, `joyselected=`; − `gamemode=`
- **2.00**: − `loadcolorsfromserver=`
- **NewWorld beta 1**: − `ekv=`
- **2.05 rev2**: + `ekv=`
- **2.1 test 2**: + `startinfullscreen=`; − `ekv=`
- **2.17.1**: + `automapping=`, `d4n=`, `kxs=`, `microactivatebyvolume=`, `microactivatekey=`, `microactivationlevel=`, `microinputdevice=`, `microon=`, `midivolume=`, `mp3volume=`, `quickplayerworlds=`, `radiovolume=`, `sfxvolume=`, `voicecodec=`, `voicevolume=`; − `musicon=`, `soundfxon=`, `usewinamp=`
- **2.22**: + `azg=`; − `d4n=`, `kxs=`
- **2.31**: + `cachefilenames=`

Sound effect names (`.wav`)

Names the executable refers to. Sounds named only by scripts or ganis are not here.

- **4th Graal alpha**: + `arrow.wav`, `axe.wav`, `bomb.wav`, `chest.wav`, `crush.wav`, `dead.wav`, `extra.wav`, `frog.wav`, `lift.wav`, `put.wav`, `sword.wav`, `swordon.wav`
- **7th Graal (beta7)**: + `water.wav`

- **8th Graal (beta8)**: + `arrowon.wav`, `compudead.wav`, `fire.wav`, `goal.wav`, `horse.wav`, `horse2.wav`, `item.wav`, `item2.wav`, `jump.wav`, `lift2.wav`, `nextpage.wav`, `sign.wav`, `steps.wav`, `steps2.wav`, `stonemove.wav`, `thunder.wav`
- **1.1.9 Gold**: − `horse.wav`, `horse2.wav`, `steps2.wav`
- **Graal3D** (separate program): none
- **1.34**: + `horse2.wav`
- **NewMain 1.42 (24)**: − `dead.wav`, `goal.wav`, `sign.wav`
- **NewWorld beta 1**: − `chest.wav`, `compudead.wav`, `frog.wav`, `item2.wav`, `jump.wav`, `stonemove.wav`, `swordon.wav`
- **2.05 rev2**: + `chest.wav`, `compudead.wav`, `frog.wav`, `item2.wav`, `jump.wav`, `stonemove.wav`, `swordon.wav`
- **NewWorld beta 2**: − `chest.wav`, `compudead.wav`, `frog.wav`, `item2.wav`, `jump.wav`, `stonemove.wav`, `swordon.wav`
- **2.17.1**: + `chest.wav`, `compudead.wav`, `frog.wav`, `item2.wav`, `jump.wav`, `stonemove.wav`, `swordon.wav`

Image names the executable refers to

- **1st Graal**: + `letters.gif`, `link.gif`, `map.gif`, `opps.gif`, `pics1.gif`, `state.gif`
- **3rd Graal**: + `heads.gif`
- **5th Graal**: + `sprites.gif`; − `link.gif`
- **5th Graal alpha**: + `map2.gif`
- **9th Graal alpha**: + `graal.bmp`, `ride.gif`, `ride2.gif`, `servant1.gif`
- **9th Graal (beta9)**: + `w-warpdoor.gif`
- **1.0 (11th)**: + `warpdoor.gif`; − `w-warpdoor.gif`
- **1.1 (15th)**: + `map3.gif`
- **1.21**: + `baddyblue.gif`, `baddydragon.gif`, `baddygold.gif`, `baddygray.gif`, `baddyhare.gif`, `baddylizardon.gif`, `baddyoctopus.gif`, `baddyred.gif`, `head0.gif`, `head1.gif`, `head2.gif`, `head3.gif`, `shield1.gif`, `shield2.gif`, `shield3.gif`, `sword1.gif`, `sword2.gif`, `sword3.gif`, `sword4.gif`, `wbomb1.gif`, `wbomb2.gif`, `wbomb3.gif`, `wbow1.gif`, `wbow2.gif`, `wbow4.gif`, `wbowi1.gif`, `wbowi2.gif`, `wbowi3.gif`, `wbowi4.gif`; − `heads.gif`, `ride.gif`, `ride2.gif`, `servant1.gif`, `warpdoor.gif`
- **1.28 beta2**: − `map3.gif`
- **1.3 test**: + `ss_arrow.gif`, `ss_bomb.gif`
- **Graal3D** (separate program): `body.bmp`, `logo.bmp`, `man.bmp`
- **1.33**: + `body.png`, `bomb1.png`, `bomb2.png`, `letters.png`, `map.png`, `map2.png`, `opps.png`, `pics1.png`, `sprites.png`, `state.png`; − `letters.gif`, `map.gif`, `map2.gif`, `opps.gif`, `pics1.gif`, `sprites.gif`, `state.gif`
- **1.34**: + `graal.png`
- **1.35**: + `graalmap.bmp`; − `head0.gif`, `head1.gif`, `head2.gif`, `head3.gif`
- **1.41 rev1**: + `graalmap.png`; − `graal.bmp`, `graalmap.bmp`
- **NewMain 1.42 (24)**: + `head0.gif`
- **2.00**: + `1hlight2.png`, `ahlight2.png`, `arrowsbox.gif`, `bluelampani.gif`, `door.gif`, `gate1.gif`, `gate2.gif`, `head19.gif`, `head24.gif`, `head26.gif`, `head27.gif`, `head495.gif`, `light2.png`, `sleepkoji.gif`, `tutorial_housegate.png`, `tutorialmap.png`, `tutorialmap2.png`
- **NewWorld beta 1**: + `bodynw.png`, `headalgo000.gif`, `newsword.png`, `pics1nw.png`, `spritesnw.png`; − `1hlight2.png`, `ahlight2.png`, `arrowsbox.gif`, `bluelampani.gif`, `body.png`, `door.gif`, `gate1.gif`, `gate2.gif`, `head19.gif`, `head24.gif`, `head26.gif`, `head27.gif`, `head495.gif`,

`light2.png`, `sleepkoji.gif`, `sprites.png`, `tutorial_housegate.png`, `tutorialmap.png`, `tutorialmap2.png`, `wbow1.gif`, `wbow2.gif`, `wbow4.gif`

- **2.05 rev2**: + `1hlight2.png`, `ahlight2.png`, `arrowsbox.gif`, `bluelampani.gif`, `body.png`, `door.gif`, `gate1.gif`, `gate2.gif`, `head19.gif`, `head24.gif`, `head26.gif`, `head27.gif`, `head495.gif`, `light2.png`, `sleepkoji.gif`, `sprites.png`, `tutorial_housegate.png`, `tutorialmap.png`, `tutorialmap2.png`, `wbow1.gif`, `wbow2.gif`, `wbow4.gif`; − `bodynw.png`, `headalgo000.gif`, `newsword.png`, `pics1nw.png`, `spritesnw.png`
- **NewWorld beta 2**: + `bodynw.png`, `headalgo000.gif`, `newsword.png`, `pics1nw.png`, `spritesnw.png`; − `1hlight2.png`, `ahlight2.png`, `arrowsbox.gif`, `bluelampani.gif`, `body.png`, `door.gif`, `gate1.gif`, `gate2.gif`, `head19.gif`, `head24.gif`, `head26.gif`, `head27.gif`, `head495.gif`, `light2.png`, `sleepkoji.gif`, `sprites.png`, `tutorial_housegate.png`, `tutorialmap.png`, `tutorialmap2.png`, `wbow1.gif`, `wbow2.gif`, `wbow4.gif`
- **2.17.1**: + `baddyblue.png`, `baddydragon.png`, `baddygold.png`, `baddygray.png`, `baddyhare.png`, `baddylizardon.png`, `baddyoctopus.png`, `baddyred.png`, `body.png`, `bshield0.png`, `emoticonmicro.png`, `g2k2pet_squirrelm0.png`, `graal2002letters.png`, `head0.png`, `head26.png`, `khairs0.png`, `khead0.png`, `klegs0.png`, `kmarms100.png`, `kmbody100.png`, `picso.png`, `shield1.png`, `shield2.png`, `shield3.png`, `sprites.png`, `ss_arrow.png`, `ss_bomb.png`, `sword1.png`, `sword2.png`, `sword3.png`, `sword4.png`, `wbomb1.png`, `wbomb2.png`, `wbomb3.png`, `wbow1.png`, `wbow2.png`, `wbow4.png`, `wbowi1.png`, `wbowi2.png`, `wbowi3.png`, `wbowi4.png`; − `baddyblue.gif`, `baddydragon.gif`, `baddygold.gif`, `baddygray.gif`, `baddyhare.gif`, `baddylizardon.gif`, `baddyoctopus.gif`, `baddyred.gif`, `bodynw.png`, `head0.gif`, `headalgo000.gif`, `newsword.png`, `pics1nw.png`, `shield2.gif`, `shield3.gif`, `spritesnw.png`, `ss_arrow.gif`, `ss_bomb.gif`, `sword1.gif`, `sword2.gif`, `sword3.gif`, `sword4.gif`, `wbomb1.gif`, `wbomb2.gif`, `wbomb3.gif`, `wbowi1.gif`, `wbowi2.gif`, `wbowi3.gif`, `wbowi4.gif`
- **2.31**: − `graalmap.png`, `shield1.gif`, `ss_arrow.png`, `ss_bomb.png`

Delphi forms (windows)

Form class names from the RCDATA resources.

- **1st Graal**: + `TADVENTURE`, `TCONNECTFRAME`, `TEDITBADDY`, `TEDITSIGNS`, `TLINKLIST`, `TLINKOPTIONS`, `TLOADLEVEL`, `TOBJECTNAME`, `TOPTIONS`, `TSHOWCODES`
- **2nd Graal**: + `TCHESTEDIT`, `TEDITPERSON`, `TPERSONACTION`
- **3rd Graal**: + `TEDITFLAGS`
- **5th Graal**: + `TMOREOPTIONS`
- **6th Graal (beta5)**: + `TOPENURL`
- **9th Graal alpha**: + `TRECONNECTFRAME`
- **1.1 (15th)**: + `TCONTROLSFRAME`
- **1.21**: + `TDEBUGGERFRAME`
- **1.28 beta2**: + `TEDITCHARACTER`, `TMESSAGEEDITOR`, `TPLAYERLIST`
- **1.32**: + `TRTFMESSAGE`
- **Graal3D** (separate program): `TADVENTURE`, `TCONNECTFRAME`, `TEDITLOOK`, `TMOREOPTIONS`
- **1.39**: + `TPLAYERLISTOPTIONS`, `TPROFILEEDIT`
- **1.41 rev1**: + `TSERVERLISTFRAME`, `TSERVERMESSAGE`, `TSHORTMESSAGE`, `TWAABOUT`; − `TRTFMESSAGE`
- **2.04**: + `TOBJECTSET`
- **NewWorld beta 1**: − `TCHESTEDIT`, `TEDITBADDY`
- **2.05 rev2**: + `TCHESTEDIT`, `TEDITBADDY`
- **2.1 test 2**: + `TEDITNPC`; − `TEDITPERSON`, `TOBJECTSET`, `TPERSONACTION`, `TSHOWCODES`
- **NewWorld beta 2**: − `TCHESTEDIT`, `TEDITBADDY`

- **2.17.1**: + `TABOUTDLG`, `TCHESTEDIT`, `TEDITBADDY`, `TLOGFRAME`, `TRPGFRAME`, `TTILESARRAYFRAME`; − `TWAABOUT`
- **2.31**: − `TCHESTEDIT`, `TDEBUGGERFRAME`, `TEDITBADDY`, `TEDITCHARACTER`, `TEDITFLAGS`, `TEDITNPC`, `TEDITSIGNS`, `TLINKLIST`, `TLINKOPTIONS`, `TOBJECTNAME`, `TTILESARRAYFRAME`

Font names in the executable

- **1st Graal**: + `Courier`, `MS Sans Serif`, `System`
- **6th Graal (beta5)**: − `Courier`
- **Graal3D** (separate program): `MS Sans Serif`, `System`
- **1.34**: + `Courier`, `Courier New`
- **2.17.1**: + `Arial`

`version-profiles/matrix/live-runs-2026-10-30.md`

Live runs of six eras under Wine, 2026-10-30

Six clients were run offline in a cloud container: 1st Graal (Sep 1998), 1.0 (10th Graal, Dec 1998), 1.1 (15th Graal, Jan 1999), 1.32 (Jan 2000), 1.41 rev1 (Nov 2000) and 2.00 (Feb 2001). Each was started in its own single-player or training mode, screenshotted, and timed while walking. No network: every client ran inside `unshare --net`. **[live]**

The screenshots, the helper scripts and the survey data are in the project folder under `graal-decompile/version-matrix/` (`shots/`, `tools/`, `survey.jsonl`). They are not in this repo.

Setup notes for the old builds

These matter for anyone running a pre-2001 client in the lab (`../../tools/lab-setup.md`, and the live-trace setup used for 2.05):

- **Keyboard.** The 1998–1.41 clients read the keyboard through DelphiX `TDXInput`, which loads DirectInput. Under Wine 9.0 no key reaches them that way. With `WINEDLLOVERRIDES="dinput=d"` the DirectInput load fails, DelphiX falls back to the Windows keyboard state, and `xdotool keydown` works.
- **A window manager.** With bare Xvfb the game window never becomes active. With `openbox` running and the Wine virtual desktop on (`HKCU\Software\Wine\Explorer Desktop=Default`, `Desktops\Default=1024x768`), the window takes focus normally.
- **Idle-driven clients.** From NewMain 1.42 the frame runs from the idle handler. Under Wine it only advances while window messages arrive, so a screen can look frozen until the mouse moves. Keep posting small mouse moves during a measurement.
- **Missing files.** 2.00's archive holds only the exe. It runs from the 2.22 install folder once `objects.dat` (from any 1.x install) is copied in. The 1.21 InstallShield groups need their heads, swords and shields placed under `levels/` before the start window will open.
- **Reading positions.** The player position is a pair of doubles in tiles in every build tried. They were found by scanning for values that rise while Right (or Down) is held and fall while Left (or Up) is held, then polled every 2 ms with unbuffered reads of `/proc/<pid>/mem`. (A buffered reader returns stale bytes and makes the game look frozen.)

Walking speed

Build	Step	Interval	Speed	Notes
1st Graal	0.5 tile	50 ms	10.0 tiles/s	x from 12.5 to 14.0 in three steps
1.0 (10th)	0.5	50	10.0	
1.1 (15th)	0.5	50	10.0	After getting out of bed and bumping the furniture, x carried a 0.025 offset (26.925) and kept it while walking. 2.05's wall rule produces such offsets (`x + 0.525`-style probe points and 0.1 nudges), so a similar rule is likely; not read
1.32	0.5	50	10.0	Measured on y
1.41 rev1	0.5	50	10.0	Measured on y. A second copy of y moves in 0.25 steps (the view or the drawn position)
2.05 rev2	0.5	50	10.0	From the 2026-10-28 live trace (`walkBC`)

So walking speed does not change from the first beta to 2.05, and the 2.17, 2.22 and 2.31 walking routines use the same half-tile step (`../../client-internals/decomp-findings-2026-10-25-m/01-walking-rule.md`).

2.00 was not timed: the training level opens with a scripted scroll that holds the player, and it did not finish within the run.

What the screenshots show

Build	Start window	In game
1st Graal	"Graal - The Adventure"; "Select Game Mode" with One Player, Two Player, One Player Online; server the loopback address, port `14801`; "Graal by Stefan Knorr, Presented by <cyberjoueur-host>"	Bedroom start. Bomb and bow icons, rupees as three digits `000`, bombs `10`, arrows `05`, three spiral hearts
1.0	"Select the game mode", grail picture, first and second player characters (Nick, Mandi), "Online At Cyberjoueur" with port 14802	Same bedroom with the blue frame. Rupees without leading zeros (`0`). The view scrolls with the player
1.1	Rider-on-horse picture, game-mode and character drop-downs (Fung Si Yan)	The player starts asleep in the left bed
1.32	As 1.1 plus Language and Nick Name fields and "Managed by: Stéphane Portha"	A/S/D buttons, the offline start level with NPCs "Stefan (Coder)" and "Kain (GP)", and a scrolling hint line ("If you have problems, then just press F1 and click on 'More options'!"). A chat line sits under the view
1.41	Language, Game mode (One Player, One Player Online, Two/Three/Four Players), Character, Nick Name; "Lead programmer: Stefan Knorr"	As 1.32 plus the AP bar under the A/S/D buttons and a small number beside the sword
2.00	Dark green "Graal 2001" window, GraalOnline logo, Language and Nickname, Training / Options / Start, "Copyright © 1998–2000 Cyberjoueur"	Training level with a parchment scroll ("Welcome to Graal!"), light effects, a minimap at the bottom left, name "Stefan" in a smaller rounded face

Under Wine the player body is sometimes not drawn (2.00) and stale strips stay after a scroll, as noted for 2.05 in `../../client-internals/live-trace-205.md`. Trust the memory reads over the pictures for anything that moves.

`version-profiles/3.04-3.153.md`

Version profiles: 3.04 (Linux) and 3.153 (Mac OS X)

Date: 2026-10-30

This note is the per-version profile for the two readable Graal 3.x clients, in the same shape as `2.22-2.31.md`. The machine-readable table is `3.04-3.153.toml`. Background on the clients themselves (Torque engine, licence file, listserver, file formats) is in `protocol/v3x-clients.md`.

Sources: the 3.04 Linux client (unpacked from UPX) and the 3.153 Mac OS X client, read statically, plus the compiled TorqueScript and prefs shipped with them. 3.04 was then run against GServer-v2 in the lab (`lab-304-linux-2026-10-30.md`). Tags: **[bin]** client binary · **[file]** shipped data · **[cap]** lab run (3.04 only) · **[inf]** inferred.

Builds

Label	Platform	Token	Windows twin
3.04 (rev 1, 3.041)	Linux i386	`G3D03014`	2.22 (`GNW03014`)
3.153	Mac OS X PPC	`G3D28015`	2.31 (`GNW28015`)

The tokens share date stamps with the Windows builds, and the wire protocol matches: gen-5 login (type 5, key byte, token, plain account and password), the same LCG cipher, framing, compression thresholds and XOR caps. **[bin]**

Login and listserver

- Listserver: id 7 `G3D30123newmain` and id 1 (account, password), sent together in one plain zlib frame; the reply is gen 5 **[cap]**. The host list comes from the signed `license/license.graal`, which names `<list-host-3x>:14922`. For Seihai the profile uses `<list-host-reborn>:14922` (supported) with port status **assumed**: no G3D token has been tried against the public listserver. **[bin][file]**
- The game login frame also carries 37 (language) and 34 for `bigmap.txt` and `minimap.txt`. The client **never sends 39** (tampercheck) and sends nothing periodic over TCP; its 5.1 s message (2.31's TCP 24) goes to other players over UDP. Then it starts UDP unless `$pref::Graal::noudp` is set, and requests its own images with 23 and 34, at most five at a time. **[bin][cap]**
- First props sent: 0, 18, 31, 33, 52, 75 (`linux`) and 76 (1250 in the lab). **[cap]**
- Server names carry a category prefix: `P` Gold, `3` 3D, `H` Hosted, `U` hidden, none Classic. **[bin]**

Client → server ids

	3.04	3.153
Sent	0–24, 26–30, 32–38, 40, 41, 43, 80, 81	the 3.04 set plus 46, 50, 93, 96–98, 152, 154–158
Complete?	yes, every game-code send site	no: 2, 3, 15, 24, 29 and 80 are assumed from 3.04 **[inf]**
Versus Windows twin	no 39 or 44; adds 36 and 81	no 39 or 44; keeps 21 (2.31 dropped it); adds 36, 50, 81 and the RC/file ids

New in 3.153: 46 ENTERLEVEL, 50 RAWDATA, 93/96/97/98 the RC file browser (up, move, delete, rename), 152 REQUESTTEXT, 154 SENDTEXT, 155/156 large-file upload start and end, 157 UPDATEGANI, 158 UPDATESCRIPT. **[bin]**

Server → client ids

3.04 handles 79 ids, 3.153 handles 88; the table is in `protocol/v3x-clients.md`. Against 2.22, 3.04 lacks 170, 177, 182 and 183 and adds 84, 101 and 102. **[bin]**

Props

	3.04	**3.153**
Own props written	0–76 except 25, 27–30, 34, 36, 50, 51	0–80 except 25, 27–30, 34, 36, 50, 51, 77
Own props parsed (PLO 9)	0–74 except 33, 50, 51, 53	0–80 except 33, 50, 51, 53, 75–77
Other players parsed	0–76	0–80
NPC props parsed	0–73 except 48–52	0–77 except 48–52
Precise move (X2/Y2/Z2, MOVE2)	no	yes

Prop 75 is OSTYPE and carries `linux` or `mac`; prop 76 is TEXTCODEPAGE. The written-prop exclusions are the same as 2.17's. **[bin]**

Player-visible features

- **3.04** start window: Nick, Language, Options, Tutorial, Account, Password, Get Account, Start. Server list tabs Graal Gold, Classic, Graal 3D and Hosted, with a direct-connect field. Options tabs Graphics, Audio, General. Player list, PMs, a profile window, and an offline mode that loads a local level. No in-client editor. **[file]**
- **GS1** (3.04): 496 registered names. Every 2.05 command is present except `putobject`, `setbackpal`, `setbody`, `setbacktile`, `drawovertrees` and `setbacktile2`. 3.153 was not counted. **[bin]**
- **3.153** adds a 3D mode (Torque `GameConnection` on UDP listed port + 1, `Graal3D` protocol 19, with missions and terrain; see `torque/wire-by-build.md`), snow and xmas themes, the RC file browser and script uploads, and server-side class scripts (131–134). The shipped prefs turn UDP off (`noudp = 1`, fixed port 14899). **[file][bin]**

Observed behaviour (3.04)

Walk speed 10 tiles per second, `push` after about 0.9 s against a wall, a 0.25 s `sword` gani, dropped items removed by the client after 8.3 s, no 11 when a bush is thrown, chat drawn as white text with a blue shadow and no bubble, and the 2.05 UDP channel unchanged (movement, chat, trigger action, a 5.1 s keepalive). Details and timings are in `lab-304-linux-2026-10-30.md`. **[cap]**

Open

- None for respawn offline: 3.04's offline mode cannot cut, lift or break classic objects at all (lab note).
- Try the `G3D30123` listserver login against the public listserver to settle the port status. This is an outward action and has not been done.
- A clean copy of the 3.151 Linux installer would give a third data point between the two.

`version-profiles/lab-304-linux-2026-10-30.md`

Lab run: the real 3.04 Linux client on GServer-v2

Date: 2026-10-30

The 3.04 Linux client (the original UPX-packed executable from the installer) was run in a cloud container: i386 libraries, Xvfb with Mesa software GL, and a network namespace with loopback only. `/etc/hosts` in the namespace pointed `<list-host-3x>` at the loopback address, so the signed `license.graal` did not need editing. The servers were GServer-v2 (with `G3D03014` added to `allowedversions.txt`) and the lab listserver, extended to accept the 3.x listserver login. One online session ran for about 5 minutes. A short offline session followed. Tags as elsewhere: **[cap]** seen on the wire or on screen in this run.

Setup, for repeating it

- Packages (Ubuntu 24.04, i386 multiarch): `libc6`, `libx11-6`, `libxext6`, `libgl1`, `libglx-mesa0`, `libgl1-mesa-dri`, `libglu1-mesa`, `libopenal1`, `libasound2t64`, plus `xvfb`, `xdotool` and `iproute2`. The client ships its own SDL 1.2 and fmod in `lib/`, and `libstdc++` is linked in.
- Run it from the install folder with `LD_LIBRARY_PATH=./lib`, inside the namespace, on an Xvfb display of 1024×768×24 with GLX. It opens at 800×600 and keeps its prefs and `console.log` under `~/.garagegames/graalexe/`.
- In the namespace, bring `lo` up before starting the servers. GServer-v2 needs `G3D03014` in `config/allowedversions.txt`. The lab listserver must accept id 7 in the first plain frame and answer in gen 5.
- The client scans its folders only at startup, so a level copied in later is not found until a restart.
- The UPX stub leaves the running process named `3`, so `pkill graalexe` matches nothing. Stop clients by pid.
- A second client needs its own `HOME` (prefs live under `~/.garagegames`) and its own X display.

Startup and listserver

- The client runs as shipped. Sound fails to open (no device) and the game carries on. The start window, server list and in-game screens match what the TorqueScript files describe. **[cap]**
- **Listserver login is one frame.** The client sends a single plain zlib frame holding both id 7 (`key`, `G3D30123newmain`) and id 1 (account, password). Static reading suggested a flush between them; the wire shows none. The lab listserver answered in gen 5 (compression byte plus partial XOR, keyed with that key), and the client showed the list and the status line. **[cap]**
- The server list shows only the tabs that have servers. A server name with no prefix appears under Classic. **[cap]**

Game login

- The first frame is plain zlib (no compression byte, no XOR) and holds four messages: 5 login (type 5, key, `G3D03014`, account, password), 37 `English`, then 34 UPDATEFILE for `bigmap.txt` and `minimap.txt`. GServer-v2 logged it as "New Client (2.22+)", version 3.041. **[cap]**
- Every later frame is gen 5. The server's first reply used bz2 (compression byte 6) and the client decoded it. **[cap]**
- First props message (id 2) after the level arrives: 0 NICKNAME, 18 STATUS, 31 UDPPORT (a random port, as `fixedport` is 0), 33 ADDITFLAGS 0, 52 PLANGUAGE `English`, 75 OSTYPE `linux`, 76 TEXTCODEPAGE 1250. **[cap]**

- Then 34 UPDATEFILE for the player's head and the two default weapon icons, and 23 WANTFILE for the player-list status icons named by server id 180. These come in two batches, which matches the five-request queue limit. **[cap]**
- **No 39 (tampercheck) in the whole session, and no periodic message over TCP.** In five minutes of mostly idle play the client sent only 1, 2, 13, 18 (GServer's movement script), 23 and 34. 2.31 sends 39 about every 3.2 s and 24 about every 5.1 s. The 5.1 s message does exist in 3.04, but it goes over UDP to other players (see the two-player run below), and this session had no other player. **[cap]**

Movement and animation

- Walking moves 0.5 tiles per 50 ms (10 tiles per second). While moving, the client sends X or Y every 100 ms. When the key is released it sends `idle` about 0.1 to 0.15 s later. **[cap]**
- **Pushing against a wall:** the `push` gani starts about 0.9 s after the player stops against a blocking tile. Releasing the key after 0.6 s did not trigger it. **[cap]**
- **Chat:** the text is drawn above the head in white with a one-pixel blue shadow down and to the right. There is no bubble or box. Tab opens the chat bar along the bottom of the screen. **[cap]**
- Key bindings (Torque `config.cs`): arrows move, A, S and D are the item keys, Q, P and M open windows, T is the microphone, F3 options, F5 load level, F7 player list, F8 server list. **[file]**

Bushes, lifting, items

- **Sword on a bush:** the client sends 34 UPDATEFILE for `sword.gani`, then 1 BOARDMODIFY (2×2 tiles), then gani `sword`. The `sword` gani lasts 0.25 s before `idle`. The server echoes 7 and adds a dropped item with 22. **[cap]**
- **The dropped item vanished after 8.3 s.** The client sent 13 ITEMDEL for it on its own, without being picked up. **[cap]**
- **Respawn online is the server's job.** GServer-v2 restored the bush with 7 BOARDMODIFY 14.6 s after the cut (its `respawntime` is 15). The client only draws what the server sends, so this lab cannot give the official server's respawn time. **[cap]**
- **Lifting a bush:** 1 BOARDMODIFY at once, gani `lift`, carry sprite 1 after 0.2 s, `carrystill` after 0.3 s, and `carry` while walking. **Throwing** sends only `idle` with carry sprite −1. No 11 THROWCARRIED was sent for a thrown bush. **[cap]**

Offline

- Offline mode (the Tutorial button) loads `levels/tiledefs/tiledefsOffline.txt`, one line: `picso.png,offlinetutorial,1,0,0`. `picso.png` is a 1024×512 sheet of terrain only (water, sand, grass, rock, snow, ice). It has no bushes, pots or liftable stones, so the tutorial levels have none either. **[file]**
- A test level (`labbush.nw`: grass, with four each of classic bush 2/3/18/19, stone 34/35/50/51 and vase 684/685/700/701 tiles) was loaded offline with F5. The objects draw wrongly (each 16-pixel cell shows a whole small object, and the bushes and stones alternate by position whatever their tile ids), the player walks through them, and the sword does not cut them. The same happened with the offline tiledefs file removed. Loading the GServer world level offline gave the same kind of broken picture. **[cap]**
- So **3.04 has no offline respawn to measure**: offline, the classic cut, lift and break objects do not work at all. Online, the server does the respawn (above). **[cap]**

Two players: the UDP channel

Two clients (separate prefs folders, one Xvfb display each) logged in to the same level. Both had `noudp = 0`, as shipped for Linux. The decoder is the 2.05 one from `protocol/udp-p2p.md`, unchanged. **[cap]**

- **Same channel as 2.05.** Each datagram is `seq` plus `GCHAR id` plus fields, XORed whole with `recipientId XOR 1`: key 3 to player 2, key 2 to player 3. `seq` counts from 0 per sender and recipient pair. There is no zlib and no length prefix.
- **Id 0, movement:** X×2, Y×2, SPRITE, STATUS (21), CARRYSPRITE (−1), and the first letter of the current gani (`W` walk, `I` idle, `S` sword). It is sent every 100 ms while moving, then once on stopping. The same position also goes to the server over TCP as props.
- **Id 15 with one byte, 9: sent about every 5.1 s to each peer.** This is the message 2.31 sends over TCP as 24 every 5.1 s with the value 9. In 3.04 it goes to peers over UDP and not to the server.
- **Id 22, chat:** the raw text. The same text also went to the server as prop 12 over TCP.
- **Id 26, trigger action:** a mouse click sent `0 0 0` (an NPC id of 0), X×2, Y×2 and the action `leftmouse`. It went over UDP only, with no TCP copy.
- In both two-player runs, one of the two clients stopped sending UDP to its peer after 11 to 20 s, and its later chat went over TCP only. The trigger for this was not traced.
- The other player's name shows under them in blue. While a client window has no focus the name reads `name (paused)`.

Not yet tested

- Pots and stones online (they need a level with them in reach and enough glove power), PMs, and the profile window.
- Why one client stops sending UDP after a few seconds.

`client-internals/live-behaviour-404.md`

Graal 4.04r2 live: the Login server, chat, movement, carrying and respawn

Date: 2026-10-30. The real 4.04r2 Windows client (token `G3D17026`) run under Wine 9 in an isolated, loopback-only lab, against the prebuilt GServer-v2 through a gen-5 logging proxy. No addresses, routine names or decompiler text.

Tags: **[live]** seen with the real client · **[bin]** read from the 4.04r2 engine · **[src]** GServer-v2 source · **[inf]** inferred.

Caveats. The lab level is `onlinestartlocal.nw` from GServer-v2's default world. The ganis, bodies and item art come from the lab server (2.22 `graal_reborn` files), so any look that is drawn by a gani is the server's, not the client's. Respawn timing is GServer-v2's (`respawntime = 15`).

The Login server never draws players

Logging in to `<login-host-4x>:14900` opens a session to a server that the client names `Login`. The level it receives is only a backdrop for the server-list window. **[live][bin]**

- When the engine sets up the player and level, it sets the draw-layer mask to 0 if the lowercase server name starts with `login`, and to all layers (0x7ff) otherwise. With mask 0 no players and no HUD are drawn. Ghost mode (S→C 170) forces its own mask (0x500) separately. **[bin]**

- The Login server hands the client over with **S→C 178 (`PLO_SERVERWARP`)**: comma text `x,<name>,<host>,<port>`. Field 0 is not used, and fewer than 4 fields are ignored, as in 2.31 (`protocol/packet-ids.md`). The client closes the Login session, logs in again to that host and port, and uses `<name>` as the server name, so the game server must not be named `Login`…. **[live]**
- In the lab, injecting `178` after the Login session's 156 made the player, the HUD (A/S/D, AP bar, hearts) and the player list appear. Before that, everything else that was tried changed nothing. That included dropping GServer-v2's id 195 answer to 157, adding `GANI0001` headers, and placing body and sprite images locally. A separate lab run found the same on the 5.00r7 client. **[live]**
- For a server emulator: either send 178 from the login session, or give the session a server name that does not start with `login`. GServer-v2 sends 178 only when its listserver forwards a server-info request.

Other login details seen live: GServer-v2 reports the client as `G3D17026 (4.042)`. The client asks for `basepackage.gupd` and `bigmap.txt` first. It answers S→C 182 with id 44 (its window titles). The saved password field was empty after a restart under Wine.

Chat

- **Tab** opens a dark red chat bar along the bottom of the game view, and Enter sends. Letter keys without Tab are game keys. **[live]**
- The text is drawn centred above the head in white bold sans, with a pure blue (#0000FF) copy 1 px down and right as a shadow. There is no bubble, frame or background, the same as the classic clients (`chat-text.md`). **[live]**
- Sent as C→S 2, prop 12 (`PLPROP_CURCHAT`). **[live]**



4.04r2 chat text over the player

Walking and pushing against a wall

- While a key is held, the client sends X2/Y2 (props 78/79) every 50 ms tick. The value is half-pixels, with the sign in the low bit. A walk step is 16 half-pixels, which is 8 px or half a tile per tick (10 tiles/s). **[live]**
- Against a wall the `push` gani starts about 0.9 s after the player stops moving, which matches 2.05's 18 blocked steps (`ledges-push-pull.md`). The client sends 157 `push` and prop 10 `push`, and goes back to `idle` when the key is let go. **[live]**
- While pushing, the client also sends a trigger action (C→S 38) `pushed,0` at a point in front of the player, about 0.55 s after the `push` gani starts and then about once a second. This is the online form of 2.05's `pushed` trigger. **[live]**
- A mouse click on the game view sends a trigger action `leftmouse` at the clicked point. **[live]**

Action key (A), sword (S), item (D)

- **A on something that cannot be lifted** (here, a green stone with the starting glove): `grab` gani (C→S 23 `grab.gani`, 157 `grab`, prop 10 `grab`). **[live]**
- **S**: C→S 34 with `sword.gani`, 157 `sword`, prop 10 `sword`, then `sword.wav`. Back to `idle` about 250 ms later. **[live]**

Cutting and lifting a bush, and respawn

- **Cut.** A sword hit on a bush sends **C→S 1 (`PLI_BOARDMODIFY`)** `x, y, 2, 2` with the four stump tiles, plus `crush.wav`. The server echoes it to the level as S→C 7. **[live]**
- **Lift.** A facing a bush sends the same 2×2 board modify, then `lift` (gani, 157, prop 10) with `item.wav` and `lift.wav`. About 300 ms later the gani becomes `carrystill`, and prop 19 (carry sprite) becomes 1 (bush). GServer-v2 then dropped an item (S→C 22, here a rupee). **[live]**
- **Respawn is the server's.** The client keeps no timer: the bush comes back when the server sends S→C 7 with the original tiles. GServer-v2 did that 14.5 s after a cut and 14.2 s after a lift, from its 15 s `respawntime` checked once a second. **[live][src]** For what the original servers did, see `board-respawn.md`.

Carrying, hiding and throwing

- **Carrying** sends `carry` while walking and `carrystill` while standing. The held object is the gani's `ATTR3` sprite, drawn last and 22 px above the body. The head and the raised hands are drawn first, so the object covers the top of the head. That comes from the gani, so another server's ganis can draw it differently. **[live]**

- **D while carrying hides the player under the object**, as in 2.05 (`carry-hide-and-glove.md`). The client sends `hiddenstill`, then `hidden` while walking, and sets status (prop 18) to 0x16, which has bit 0x02. Only the bush is drawn, and the player is not. Hidden walking moves 1.6 px per tick on average (steps of 4 and 2 half-pixels), which is 0.2 of a normal step, the same factor as 2.05. **[live]**



- **A while carrying throws**: C→S 11 (`PLI_THROWCARRIED`) with an empty payload, then `idle`, and the carry sprite is cleared. **[live]**

Pots, stones and black stones

For these, two pots, a stone and a black stone were placed on the lab level, and the lab account was given glove power 3. The tile numbers are the ones in `board-respawn.md`. **[live]**

Object	Glove needed	Board after lifting	Carry sprite (prop 19)	Item drop (GServer-v2)	D hides	Thrown	Back after
Bush (2)	1	677 678 / 693 694 (stump)	1	sometimes (a rupee seen)	yes	leaves, `crush.wav`	14.2 to 14.5 s

Object	Glove needed	Board after lifting	Carry sprite (prop 19)	Item drop (GServer-v2)	D hides	Thrown	Back after
Pot (684)	1	1770 1771 / 1786 1787 (empty base)	5	a heart, every time	**no**	shards, `crush.wav`	15.0 s
Stone (34)	2	1834 1835 / 1850 1851 (hole)	3	none	yes	fragments, `crush.wav`	15.0 s
Black stone (990)	3	1834 1835 / 1850 1851	GCHAR byte 0xe9 (201), not decoded	none	yes	`crush.wav`	14.3 s

- With glove 1, A on a stone plays `grab` instead, as in 2.05. **[live]**
- A pot gives `item.wav` with the lift, because the server drops its heart at the same moment. Stones only give `lift.wav`. **[live]**
- The thrown object breaks about 250 to 350 ms after the throw, which is when `crush.wav` is sent. No board change or other packet marks where it landed. **[live]**
- Coming out of hiding with A sends `carrystill` and status 0x14, which clears bit 0x02. A second A throws. S throws straight from hiding. **[live]**
- Every respawn is a single S→C 7 with the original 2×2 tiles. The 14 to 15 s spread is GServer-v2's 15 s timer, checked once a second. **[live][src]**



pot: lift, D (no hide), throw and shards



black stone: carried, hidden (D), thrown, gone

Hidden state survives a reconnect

The client was closed while the player was hidden. GServer-v2 saved `ANI hiddenstill` and `STATUS 22` to the account, but nothing was being carried. On the next login only the shadow was drawn, and the player came back on the first step. A server should clear the hidden status bit and the hiding gani when it saves or loads an account. **[live]**

4.20r8 (the packet-logger build)

The 4.20r8 engine survives only in a third-party packet-logger build (`protocol/v4x-clients.md`). Its logging hook calls fixed Windows XP SP2 addresses, so it crashes anywhere else. In a local copy, the eight original bytes at the hook's only call site (the end of zlib's inflate) were put back, which turns the build into a plain 4.20r8 engine. That copy ran under Wine with the 4.04r2 data files. **[live][bin]**

- GServer-v2 accepts it with token `G3D20126`, and it goes through the same Login server and S→C 178 handover. **[live]**

- Chat: the same white bold text with a blue shadow, and the same dark red Tab bar. After Enter, the bar stayed open with the sent text in it. **[live]**
- Walking is 8 px per tick. The `push` gani starts 0.85 s after the last step, and the `pushed` trigger action follows 0.55 s later and then once a second, as in 4.04r2. **[live]**

Live run 2026-10-31: 4.04r2 and 4.20r8 next to Seihai

A second lab, run in a cloud container: Wine 9, Xvfb, a loopback-only net and mount namespace, the seihai-lab server pack with stock gs2lib, and a proxy on 14900 that injects S→C 178 `0,Lab Server,<loopback>,14900`. The 4.20r8 engine was restored from the packet-logger copy by putting back the 8 bytes at its hook: `GraalEngine.dll` offset 0x1a3ae3, `e9 f3 cc 08 00 90 90 90` back to `5f 5e 5d b8 01 00 00 00`. **[live] [bin]**

- **After the login, both builds send:** 37 (language); 23 `basepackage.gupd` twice; 23 `bigmap.txt` (in the Login session only); 157 `idle`; 152 `GraalEngine,pmguilds,""` and `GraalEngine,pmservers,""`; 154 `GraalEngine,lister,options,globalpms=true,buddytracking=true,showbuddies=true` and `GraalEngine,lister,verifybuddies,1,0`; **46**; 44 (window titles); then 2 (own props). GServer-v2 logs the 152 as RequestText and the 154 as SendText. **[live]**
- **Walking** is the same in both builds: 2 with GANI `walk` and SPRITE on the first step, then X2 (78) or Y2 (79) alone every 50 ms, then `idle`. X/Y (15/16) are never sent. **[live]**
- **Script flags.** A GS1 probe weapon ran these lines on `playerenters`:
  - `setplayerprop #c`
  - `setstring gr.probe,1`
  - `setstring client.probe,2`
  - `set probeflag`

  The 4.04r2 client sent only C→S 18 `client.probe=2`. The lab's `-gr_movement` weapon (`setstring gr.x` every 0.05 s) sent nothing either. So the client runs GS1 weapon scripts but sends only `client.` string flags to the server. **[live]**
- **Chat bar.** The chat bar stays open after Enter in 4.04r2 as well. Tab closes it and sends an empty prop 12. Esc opens "Do you really want to quit Graal?". **[live]**
- **Bow.** D with the bow sends 157 `shoot`, prop 10 `shoot,wbow1.png,arrow.wav`, and C→S 9 (arrow). With bombs, D sends C→S 4. **[live]**
- **In the Login session the game works.** Walking, chat and the sword all send as normal. Only the drawing is cut, as described above. **[live]**
- **Ids.** C→S ids seen: 2, 4, 9, 23, 24, 34, 37, 38, 44, 46, 152, 154 and 157. S→C ids: the 26 from the first run plus 45. **[live]**

Seihai's write-up, with the fixes this run led to (X2/Y2 walk, `client.` flags only), is in seihai `documentation/v4-profiles.md`. The logs are in the project folder under `v4/lab-4x-live-2026-10-31/`.

Third run, 2026-10-31: update packages, frames, focus and the walk in detail

The same lab, rebuilt in a fresh container (stock 4.04r2, seihai-lab pack with stock gs2lib, `warpproxy.py` on 14900). Logs are in the project folder under `v4/lab-4x-after-login-2026-10-31/`. **[live]**

- **Update packages.**
  - With no copy, the client asks `basepackage.gupd` with C→S 23, twice.
  - With the server serving it, the client saves it in `updatepackages/basepackage.gupd`. On the game server it also saves a copy in `updatepackages/<server name, space as %032>/` (`Lab%032Server`).

    - On the next run of the program it sends **C→S 47 (`VERIFYWANTSEND`)** twice instead: a GUINT5 CRC-32 of the cached file, then the name. For `GRPKG001` and a newline that is `! MCt'basepackage.gupd`, CRC 0x15a8ea07.
    - Within one run it keeps asking with 23 (its in-memory copy is not checked). This is the same rule 5.00r7 follows (`#145`).
- **`bigmap.txt`** is asked once per run of the program. It is asked on the Login server and not after the warp.
- **The after-burst messages** come in four frames a few milliseconds apart:
    - 23 `idle.gani` and 157 `idle`
    - the head image, 152 ×2 and 154 ×2
    - 46
    - 44, the default weapons' images, 2 (own props), 24, then 34 `pics1.png`

  157 is `PLI_UPDATEGANI`: a GUINT5 checksum (0, since the client has no copy), then the name. It goes with each 23 for a GANI (`idle`, `walk`, `sword`, `shoot`), once per connection.
- **Nickname.** Prop 0 in the own props is the start screen's Nickname field (`Real404`), not the account.
- **Walk.**
    - The first step sends `GANI walk` and SPRITE, then the moving axis (X2 or Y2).
    - Each 50 ms tick sends only the axes that moved: both X2 and Y2 on a diagonal (`n*@o)@`), and only the first axis in the very first message.
    - `idle` goes out alone one tick (50 ms) after the last step.
    - Every step is 8 px.
- **Focus.** Losing the window focus sends C→S 2 STATUS 21. Getting it back sends 20. No position goes with either. **[live]**
- **24** is `GCHAR 9` every 5.142 to 5.151 s while alone in the level. A tick can carry it in a walk frame.

Seihai's 4.x profiles now send all of the above except the GUI image checks (squirelworks/seihai#295).

Not covered yet

- The client's own ganis, from a 4.x-era server rather than the 2.22 world, to compare the look.
- Signs (lift entry 1).
- Carrying and hiding on 4.20r8. Only chat, walking, pushing, the sword and items were repeated there.

`version-profiles/lab-mac-tiger-2026-10-31.md`

Lab run: the real Mac 4.20r6 and 3.153 clients on Mac OS X 10.4 in qemu

Date: 2026-10-31. Tags as elsewhere: **[cap]** seen on the wire or on screen in this run. Raw captures (proxy logs, pcaps, screenshots, the qemu driving scripts and `aglshim.c`) are in the project files under `v4/lab-mac420r6-2026-10-31/` and `v3/lab-mac3153-2026-10-31/`.

4.20r6

The real `Graal.app` 4.20r6 (upload sha256 e1b60bcc…1d9d, executable unmodified) ran on Mac OS X 10.4 Tiger (retail DVD, build 8A428, archive.org `macosx10.4tigerretaildvd`, MD5 28b3352b8b125469f23ed9c3e386f4a3, matches archive.org) in `qemu-system-ppc 8.2.2 -M mac99,via=pmu`, in the cloud container. Server: `seihai-lab/servers/seihai-lab-servers.tar.gz` (GServer-v2 efb5b90) started with `./run-lab.sh stock`, `G3D27076` added to `allowedversions.txt`, the gserver's `listport` and the lab listserver moved to 14899. `scripts/warpproxy.py` (the 6.x lab proxy, warp host changed to the qemu host's

address as seen from the guest) listened on the login server port; the guest's `/etc/hosts` pointed the built-in login server names at the qemu host, as in the 4.04r2 Wine lab. The lab account, throwaway password.

Login line (captured, `run2.pcap`, `proxy.log`)

```
%  <key>  G3D27076  <GSTR account>  <GSTR password>  mac,"",a87b712247c6c9068448ba8c2f68a24b,""\n
```

- id 5, then the key byte, the token `G3D27076`, account and password as GSTR, then the comma list.
- Four comma fields: `mac`, `""`, a 32-hex MD5, `""`. The same line was sent on both connections (Login server and after the 178 warp).
- Field 2 (the binary's hash of the primary Ethernet MAC from IOKit) came out **empty**, as `""`. The guest has a qemu `sungem` NIC with a MAC, so the client's IOKit lookup found nothing it accepts there. Whether a real Mac fills it is not shown by this run.
- Field 3 (hash of `stat("/etc")`) is a plain lowercase 32-hex string.
- Field 4 is `""`, as the static read said.
- Seihai's 4.20r6 row sends `mac,"","",""`. That matches the shape and the empty fourth field. The real client fills field 3.

Client sends seen (C→S ids)

Session 1 (Login server, before the warp) and session 2 (after the 178 warp to `0,Lab Server,HOST,PORT`):

id	what
37	`English`
23	`basepackage.gupd` twice, `bigmap.txt` (session 1 only), then gani/image/font names (`+.gani`, `wbomb1.png`, `wbowi1.png`, `chest.png`, `helveticabd.ttf` `helvetica.ttf`, `arialbd.ttf`, `state.png`, `sword1.png`, `idle.gani`, `steps.wav`, `sword.gani`)
157	`+`, `idle`, `sword` (gani change)
46	`!`
44	empty (answer to 182)
2	props: first `00 "" / 18=20 / 31=0 / 33=0 / 52 "English" / 75 "mac" / 76=0` (decoded with GServer-v2's prop sizes; raw in `proxy.log`); walking **78/79 (X2/Y2)**, prop 10 gani `walk`/`idle`/`sword`, chat prop 12
34	`&w6\x87$pics1.png`, `&w6\x87#defaulttextedit.png`
24	`)` repeated

So the ids seen are 2, 23, 24, 34, 37, 44, 46, 157. Against `4.04-4.20.toml`'s 4.20r6 `pli_sent` (4.20r8's list plus 162), **34 and 46 are missing from that list** but the real client sends both. Seihai's chain already takes 34 and 46 as sent (from 3.153), so Seihai's set covers what was seen.

Not seen: id 18 (flag set). Seihai's 4.20r6 lab run sent `18 gr.x=30` / `gr.y=30.5` from the lab's `gr_movement` weapon (`setstring gr.x,…`); the real client ran the same level and weapon and sent no 18 in either session.

Window title stayed empty; the level, HUD (A/S/D, AP) and chat text (white with a blue shadow) drew. The player sprite did not draw: this server pack has no body/head art (the 4.04r2 lab copied 2.22 `graal_reborn` art for that).

Server ids received (session 2)

4, 6, 7, 9, 10, 14, 25, 30, 33, 34, 39, 41, 42, 43, 47, 82, 100, 101, 168, 182, 190, 194. The client stayed connected through all of them, including 41 (no case in the static read).

Notes and caveats

- **qemu has no 3D**: the client asks AGL for `RGBA, DOUBLEBUFFER, ACCELERATED (0x49), NO_RECOVERY (0x48), DEPTH 32/16` (static: the engine's pixel-format setup in the Mac binary) and stops with "Could not find valid pixel format" (`console-unshimmed-run.log`). `aglshim.c`, built in the guest with the DVD's gcc 4.0, is injected with `DYLD_INSERT_LIBRARIES` and drops only those two attributes, so Apple's software renderer is used. The executable and its signature files are untouched.
- First try was on the beta4 pack: GServer-v2 beta4 **segfaulted** (strlen on NULL in libc) on this login (`proxy-run1-beta4-crash.log`, `run1-beta4-crash.pcap`). The efb5b90 stock pack accepted it.
- One stray chat line (`curl …`, typed into the game by mistake) is in `proxy.log` at the end; ignore it.
- `scripts/` has the qemu driving scripts (monitor screendump, closed-loop mouse for Tiger's relative USB mouse, typing).

3.153

The real Mac `Graal.app` 3.153 (from the `GraalSetup.dmg` upload, executable sha256 6eb0a1f9…83fc, unmodified) ran on the same Tiger 10.4 qemu guest as the 4.20r6 run (the 4.20r6 section above), with the same `aglshim.dylib` (drops only AGL_ACCELERATED and AGL_NO_RECOVERY so the software renderer is used). Servers: `seihai-lab-servers.tar.gz` (GServer-v2 efb5b90) with `./run-lab.sh stock`, `G3D28015` added to `allowedversions.txt`. The lab listserver (`listserver-lab-qemu.py`) is the pack's, with two lab edits: its gserver port moved, and the server entry's address is given as `HOST:PORT` of the qemu host and the logging proxy in place of the loopback address. The guest's `/etc/hosts` pointed the built-in listserver names at the qemu host. `warpproxy.py` ran with `NOWARP=1` in front of the gserver (no 178 injection). The lab account, throwaway password.

Listserver

`listserver.log`: one connection to 14922, id 7 in gen 5 with token `G3D30123` and list `newmain`, then id 1 with the account. The client showed the server list (tab "Hosted") and connected from it.

Game login (captured)

First frame (plain zlib), four messages in one frame:

```
5   %<key>G3D28015<GSTR account><GSTR password>
37  English
34  #\x9dyO$bigmap.txt     34  #\x9dyO$minimap.txt     (then both again)
```

No platform/machine comma list on the login line (unlike 4.x). Same layout as the 3.04 Linux lab capture, with the token `G3D28015`.

First props (id 2): `0 "unknown"` (the start window's nickname field), `18=20`, `31=0`, `33=0`, `52 "English"`, **`75 "mac"`**, **`76 = 1250`** (GINT bytes `20 29 82`). Seihai's 3.153 row took OSTYPE mac from the binary and the code page 1250 from 3.04; both are now seen on the wire.

Walking: X2/Y2 confirmed

Walking sends props **78 (X2) and 79 (Y2)**, never 15/16 (old X/Y):

```
2  *$walk1#n'p          (gani walk, sprite/dir, X2)
2  n'\x80  n'\x90 ...    (X2 every ~150 ms)
2  *$walk1"o)0  o)@ ...  (Y2)
2  *$idle
```

So Seihai PR #294 (3.153 profile to X2/Y2) is right.

Non-`client.` flags

The lab level has the `gr_movement` weapon (`//#CLIENTSIDE … setstring gr.x,…; setstring gr.y,…; timeout=0.05`). In this session the client sent **no id 18 at all** (0 in `proxy.log`), while walking for several seconds. The 3.04 Linux lab run did send 18 for this script, and Seihai's 4.20r6 lab run sent `18 gr.x=30`. The real 4.20r6 Mac client also sent none. What this run cannot tell apart: 3.153 not sending non-`client.` flags, or the script not setting them in this build. The real client shows the same nothing either way.

Other sends

C→S ids seen: 2, 23, 24, 34, 46, 157 (plus 5 and 37 in the first frame). - 34 UPDATEFILE for `wbomb1.png`, `wbowi1.png`, `defaultbutton.png`, `playerlist_icons.png`, `walk.gani`, `sword.gani`, `steps.wav`, `steps2.wav`, `sword.wav` (3.x uses 34 where 4.20r6 used 23 for ganis and sounds). - 157 with the gani name (`walk`, `sword`), 46 `!`, 23 `chest.png` and others. - 24 `)` about every 16 to 18 s (614.06, 632.99, 650.49, 666.74, 683.90). - Sword: 34 `sword.gani`, 157 `sword`, prop 10 `%sword`, 34 `sword.wav`, then `idle` 0.75 s later. - Chat: prop 12 (`,/hello from 3153`), drawn white with a blue shadow above the head.

Server ids received: 4, 6, 7, 9, 10, 14, 25, 30, 33, 34, 39, 41, 42, 43, 47, 82, 100, 101, 168, 182, 190, 194. The player sprite drew (3.153 ships its own body/head art).

`version-profiles/lab-v5-2026-10-30.md`

Lab run: real 5.00r7, 5.12 and 5.24r4 clients on GServer-v2

Date: 2026-10-30

Three real Graal 5 Windows clients were run under Wine 9.0 on Xvfb, against the prebuilt GServer-v2 lab server, in a network namespace with loopback only. The clients were the full 5.00r7 install (with the private folders left out), the original MoleBox-packed 5.12 setup executable, and the MoleBox-packed 5.24r4 executable. A logging

proxy between client and server decoded gen 5 in both directions. A second version of the proxy could also rewrite or add server messages.

Tags: **[cap]** means seen on the wire or on screen in this run. **[bin]** means read from the client binary. Times come from the proxy log, which has millisecond timestamps. A tick is 50 ms.

Setup, for repeating it

- In the namespace, `/etc/hosts` points `<login-host-4x>*` and `<list-host-3x>*` at the loopback address. The stock `license/license.graal` (host `<login-host-4x>`, port 14900) then reaches the proxy on 14900, which forwards to GServer on 14901.
    - The 5.00r7 folder came with a `license.graal` that had been edited to point at a private LAN host. The stock file from the 5.12 bundle replaced it.
- GServer-v2 needs the tokens in `config/allowedversions.txt`: `G3D22067` (5.00r7), `G3D14097` (5.12), `G3D15010` (5.24r3/r4), `G3D26090` (5.30r3/5.31r4).
- **Stock gs2lib does not know `G3D15010`.** It treats 5.24 as a pre-2.1 client, so file messages come in the old layout. 5.24 then tries to save PNG data as the file name ("Couldn't open …1.png‰PNG… for writing"). In a lab copy of `libgs2lib.so`, the `G3D26090` table entry was renamed to `G3D15010`. With that copy, 5.24r4 downloads files normally. **[cap]** Since 2026-10-31 the proper fix is `tools/lab-servers/gs2lib-CLVER_5_24.patch`, which gives 5.24 its own slot and leaves `G3D26090` in place. The rule (layout chosen by the server's token table) is in `protocol/file-transfer.md`.
- Wine:
    - Under Wine, 5.00r7 and 5.12 draw through Direct3D 7, and 5.24r4 draws through OpenGL. **[cap]**
    - Setting wined3d's renderer to Vulkan gave a black screen. The default GL renderer works. **[cap]**
    - There is no window manager, so `xdotool windowfocus` on the game window is needed before keys reach it. The client also pauses while unfocused. **[cap]**

Login mode: why nothing is drawn at first

- **Fact.** The client treats the server it reaches through `license.graal` as the **Login** server. The console says "Loading tile definitions for server Login", and the window title is " L o g i n ".
- **What it hides.** In Login mode the client draws the level tiles, chests and image NPCs, but no characters (neither its own player nor other players) and no default HUD. On the real service, that server is only the backdrop for the scripted server list. **[cap]**
- **Same check in 4.04.** The v4 lab found the same rule in the 4.04 engine. If the current server name starts with "login" (case-insensitive), the draw-layer mask is 0 instead of the full mask.
- **What did not fix it.** None of these changed anything:
    - adding body, head, sword and shield images and ganis locally;
    - a fresh file-name cache;
    - a gani script reply;
    - a curlevel prop for the player itself;
    - removing the colours prop;
    - a different Wine renderer.
- **The fix is S→C 178 (`PLO_SERVERWARP`).** Send it after the login burst. Here it was sent right after S→C 182, with the text `0,Lab Server,<loopback>,14900` (host field = the loopback address).
    - The client closes the connection and logs in again with a new key, to the host and port in fields 2 and 3.
    - The second connection is named after field 1. The window title becomes " L a b S e r v e r " (5.24r4: "Lab Server").

    - On that connection the client draws:
        - the player, with shadow, head, body, sword and shield;
        - other players, with their nicknames;
        - hearts, the A/S/D item boxes, the counters, the AP bar;
        - the native player list window. **[cap]**
- **GServer-v2 and 178.** GServer-v2 sends 178 only when a listserver answers its server-info request, so the lab proxy injected it. The same warp works for 5.31r4 and 6.03r7 (the "Play v5 and v6 clients" lab).
- **Tile definitions after the warp.** The client asks for `tiledefs<servername>.txt`, with the space escaped as `%032`.
- **Implication for a server emulator.** A v5+ client must end up on a server whose name does not start with "login". Otherwise the player never appears.

Message flow seen in this run

- **Login.** The login frame and the order of the server's login burst are as described in `protocol/v5x-clients.md`. **[cap]**
- **Frame types the client accepts.** The rewriting proxy re-encoded every server frame. With every frame sent as zlib (type 4), the client disconnected after a few frames. It stayed connected once payloads of 55 bytes or less went as type 2 (plain, XOR over up to 12 dwords) and larger ones as zlib. Payloads over 8 KiB went as bz2 (type 6). That 55-byte limit is the one GServer uses. **[cap]**
    - Only the combination was tested. The exact rule the client applies is not known.
- **Movement.**
    - While walking, the client sends a props message (C→S 2) **every tick**. It carries only the coordinate that changed: X2 (prop 78) or Y2 (prop 79). The value is pixels × 2, with the low bit as the sign, so positions go out in whole pixels.
    - The gani change goes in the same message as the first step (`*` walk, then `n/o`).
    - 5.24r4 also adds prop 17 (sprite/direction) to gani changes. 5.00r7 and 5.12 do not. **[cap]**
- **Files.**
    - A gani is checked with C→S 34 (UPDATEFILE), or with C→S 23 (WANTFILE) when it is not cached.
    - Then comes C→S 157 (the gani name and a 5-byte checksum). The server answers with S→C 195 (gani name and attribute text, for example `SETBACKTO idle` for `sword`). **[cap]**
    - 5.12 and 5.24r4 fetch `sword.wav` and `lift.wav` from the server with WANTFILE the first time they are needed. 5.00r7 has them locally. **[cap]**
- **Pushing into an object.** While the push pose lasts, the client sends C→S 38 with the text `pushed,0` and the object's position. The first goes about 550 ms after `push` starts, then one every second. **[cap]**
- **Cutting and lifting a bush.**
    - Both send C→S 1 (BOARDMODIFY) with the 2×2 stump tiles. GServer echoes it to everyone as S→C 7. **[cap]**
    - Lifting sets prop 19 (carry sprite) to 1.
    - Throwing sends C→S 11 (no payload) and sets the carry sprite to −1. **[cap]**
- **Respawn is the server's job.** GServer-v2 restored the bush with S→C 7 exactly 15.0 s after the cut, which is its `respawntime = 15`. The client does not put tiles back by itself online. The respawn time a real 5.x server used was not observed. **[cap]**
- **Periodic messages.** C→S 24 about every 5 s, and S→C 42 (world time) every 5 s. **[cap]**
- **Clientside scripts.** NPC scripts arrive as source text in S→C 3 (with `§` for line breaks), not as bytecode. Clientside NPC scripts were only tried while the clients were still in Login mode, so whether they run was not settled. **[cap]**

Timings

All three builds gave the same numbers.

action	5.00r7	5.12	5.24r4
walk	0.5 tile (8 px) per tick, 41 updates in 2.0 s	same	same
first step after key down	about 1 tick (16–54 ms)	same	same
key released → `idle`	next tick (38 ms)	same	same
last refused step → `push`	908 ms (18 ticks)	902 ms	902 ms
`sword` → `idle`	248 ms (5 ticks)	243 ms	246 ms
`lift` → `carrystill`	304 ms (6 ticks)	not run (bush already cut)	299 ms
carry walk	0.5 tile per tick, `carry`	—	same
hidden walk (D while carrying)	—	—	35 px in 22 ticks = 0.1 tile per tick, `hidden` / `hiddenstill`
blocked against a corner	small x nudges of 1–2 px per tick toward the gap	—	—

These match the 2.05 timings in `client-internals/player-mannerisms-live.md`: push on the 18th blocked tick, sword 5 ticks, lift 6 ticks, hidden 0.1 tile per tick. **[cap]**

How it looks

- **Chat text** (another player's chat, seen from 5.12):
  - It is drawn above the head, centred, as white bold sans-serif text with a blue drop shadow down and to the right.
  - The shadow colour is mostly RGB (88, 88, 255), with some pure (0, 0, 255).
  - There is **no bubble box**. **[cap]**
  - The nickname is drawn the same way under the player.
  - A player whose window is inactive shows as `name (paused)`.
  - The client's own nickname is not drawn under its own player.
  - Shot: `client-internals/mannerisms-shots/v5-512-chat-text-zoom.png`.
- **Chat input needs the server.** When it decides whether keyboard input belongs to a text field, the engine looks for a GUI control named `ChatBar`. **[bin]**
  - On a bare GServer-v2 there is no such control. Typing, Tab and Enter do nothing, and no chat message is sent. **[cap]**
  - The chat bar is therefore part of the server's scripted GUI, not of the engine. The chat text above was injected as prop 12 through the proxy.
- **Carrying and hiding:**
  - A carried bush is drawn over the top of the head. Facing down, the face stays visible below it (`v5-500r7-carry-down.png`).
  - In `hiddenstill`/`hidden` only the bush is drawn, at the player's feet, and the player is not drawn at all (`v5-524-hold-hide-hidden.png`). **[cap]**
- **HUD:**
  - 5.00r7 and 5.12 label the item boxes A, S and D and print the sword level next to the sword.
  - 5.24r4 draws the same boxes without the letters and without the level number (`v5-hud-512-vs-524.png`).
  - **Possibly Wine-specific.** 5.24r4 is the one build that draws through OpenGL here. It may simply not be drawing that text under Wine, so confirm this on real Windows before using it in a profile. **[cap]**

- **Player list.** The player list is a native Windows window, titled "Players: N online", with tabs and the groups Staff, Buddies, Guild, Normal, Ignore and Offline. 5.00r7 and 5.12 opened it at login. 5.24r4 did not. **[cap]**
- **Tile graphics.**
  - In Login mode the level used one tile set.
  - After the warp, 5.00r7 used its newer default tile graphics. Bushes became the red-and-white flower bushes, and stones became pale green blocks (`v5-500r7-after-warp.png`).
  - The client asks for `tiledefs<servername>.txt` after the warp, so the change of art probably comes from tile definitions chosen per server name. **[cap]** for the request, the cause is an inference.

Not covered here

- Pots and stones: the test level has none. The respawn rule above (the server restores the tiles after its `respawntime`) applies to them as well; their per-object timing was not measured.
- 5.30r3 and 5.31r4 were run by the "Play v5 and v6 clients" lab, not here.

Lab files

The project files hold the lab tools and the raw logs, under `graal-decompile/v5x/lab/`:

- the proxy and the rewriting proxy (`g5mitm.py` with `rules.py`);
- the input driver and the measurement suite (`drive.py`, `suite.sh`, `analyse.py`);
- the raw proxy logs and the full-size screenshots.

Topics

Movement, then combat, then the HUD and presentation, then files and the session. A feature-spec note sits with the topic it describes. The feature-spec index is first.

`client-internals/feature-specs/README.md`

Seihai laundry list: the 14 "Not yet discussed" topics

Written 2026-10-24 (Asia/Tokyo, JST). Sources checked: Graal_analysis `8da6800` (main), Seihai `06aacfd` (main, 2026-10-24 03:23 JST), GServer-v2 source (local checkout), graal-serverlist source, the offline lab, and the public-server report of 2026-10-24 17:56 JST (structure only; no addresses copied). Nothing was run against the lab or any public server for this pass. No decompiler output, addresses or auto-generated names appear in these files. GServer behaviour is described in my own words with function names only.

Sufficiency table

"Enough" means Claude can implement the topic against the lab with what is already written. Value = worth to a playable client (1-5). Effort = work (1-5, 1 = a few hours). Score = value / effort.

#	Topic	Existing material enough?	Value	Effort	Score	Rank	NEEDS
13	Ban and kick handling	Yes	3	1	3.0	1	nothing

#	Topic	Existing material enough?	Value	Effort	Score	Rank	NEEDS
12	Server MOTD and rules	Yes	3	1	3.0	2	nothing
04	Death, respawn, regen	Yes (server side and 2.05 client side, from the 2.05 decompile)	4	2	2.0	3	nothing; open: the "same name group" respawn-point rule, 1.39 death handler
08	Tamperchec k 39 (+ process list 182)	Yes (constants for 2.17, 2.22 and 2.31 captured; derivation decoded)	2	1	2.0	4	nothing. Derivation settled (two chained CRC-32 words over the client's own image; per-build constants for 2.17, 2.22, 2.31). Left: whether any server enforces it (cannot be settled offline)
03	Inventory, items, chests	Yes	5	3	1.7	5	nothing
14	Friends, ignore, mute, status	2.05 (local files) yes; 2.22 buddy/ ignore local and mute payload captured; listserver buddy requests unseen	3	2	1.5	6	only if buddies should go through a listserver: a capture of `verifybuddies`/ `addbuddy`/`deletebuddy` (needs a database-backed listserver or another trigger)
02	Weapons and combat	Yes for the protocol; 2.05 sword hit box and swing sender known from the decompile	5	4	1.25	7	nothing for interop; open: whether the attacker also sends HURTPLAYER (26) on a sword hit
06	GMAP overworld and map links	Server side, file format and 2.22 client crossing behaviour yes (lab capture); 2.31 crossing captured (lab, batch b2)	4	4	1.0	8	nothing for 2.22. 2.31 crossing now captured in the lab (cell prop plus wrapped coordinate, PLI 46 before it); left, thin evidence: the Z2 prop and the overview layout on the map screen

#	Topic	Existing material enough?	Value	Effort	Score	Rank	NEEDS
11	Emotes and non-walk GANIs	Yes (2.22+ names are strings; 2.05 wire sprite formula and Ctrl+letter lookup known from the decompile)	3	3	1.0	9	nothing to build; open: how other players see a key emote, swim on the wire (none in 2.05)

01 | Sound and music | Yes for WAV effects | 3 | 3 | 1.0 | 10 | nothing |
09 | In-game options | Yes | 2 | 2 | 1.0 | 11 | nothing |
10 | Right-click context menu | Yes (2.05 F7 popup order and rules known from the decompile; no world right-click exists) | 2 | 3 | 0.7 | 12 | nothing; open: 1.39 popup, History window contents |
07 | UDP peer channel | Yes | 1 | 3 | 0.3 | 13 | nothing (UDP is optional; TCP fallback covers it) |
05 | Player save and load | Yes (server-owned) | n/a | 0 | n/a | n/a | nothing, nothing to build |

Suggested build order (dependencies, not just score): 13, 12, 08 (opt-in), 03, 02 with 04 together, 14 (ignore first), 11, 01, 09, 06 (after the lab capture), 10, 07.

Items that truly need a new decompile or capture

1. ~~Decompile, 2.05: the death handler (topic 04).~~ **Filled** by the 2026-10-24 2.05 decompile (timings, input lock, hearts by AP, destination, no respawn packet, CLAIMPKER from the victim). Left: the "same name group" respawn-point rule.
2. ~~Decompile or capture, 2.05: the sprite-index to animation table and the Ctrl+letter emoticon mapping (topic 11).~~ **Filled**: wire sprite formula and step table, and the Ctrl+letter lookup (a local prefix match on `emoticon_<letter>`, no packet). Left: how other players see an emote, and swim on the wire (none in 2.05).
3. ~~Decompile or screenshot, 2.05: the F7 popup and right-click handler (topic 10).~~ **Filled**: item order Message, History, Buddy, Ignore, Profile; `TPersonAction` is the NPC script editor, not a player menu; no world right-click exists.
4. ~~**Lab capture, 2.22**: crossing a gmap edge in a 2x2 gmap (topic 06).~~ **Filled** (see `06-gmap-overworld.md`; 2.31 crossing also captured, see `../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`).
5. **Still open: lab capture, 2.22, with an extended listserver stub**: add/verify/delete buddy and the playerlist Mute item (topic 14).
6. ~~**Lab captures**: the id-39 constant for 2.17 and 2.31 (topic 08).~~ **Filled**: constants captured for 2.17, 2.22 and 2.31, and the builder decoded (`../decomp-findings-2026-10-24-b/01-tamper-check-id39.md`). Nothing left to decompile; only whether a server enforces the value is unknown.
7. ~~Optional, feel only: the 2.05 sword hit box (topic 02).~~ **Filled**: the victim's client evaluates the hit (reach 0 to 4 tiles, lateral under 2, AP 100 immune). Left: when 2.05 sends HITOBJECTS in a swing and the real damage formula.

Settled index (cross-cutting items, status as of the lab batch c, 2026-10-24)

These are protocol items from the laundry triage that are no longer "needs decompile or capture". Detail and provenance are in the linked notes; nothing here is new evidence.

Item	Status	What is settled	Still open	Where
`PLI_ENTERLEVEL` (46), 2.31	**Settled** (lab, real client)	Payload is always `GCHAR 1`. Sent on every level entry (login, each gmap edge crossing, doors and edge links, server-side warps), just before the first props of the new level: in the same frame as a cached-level link request, otherwise in the first frame after the server's level answer. 2.31 does not re-send id 39 at any of these. 2.22 never sends 46 (0 of 8 transitions) but sends 39 after each client-initiated link request. Not sent by 2.17.	Script-initiated warps (`setlevel2`) not captured.	`../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`, `../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`, `08-tampercheck.md`
`PLO_SERVERWARP` (178), 2.31	**Settled** for the client side (lab: one real chain, field-order cases synthetic)	Four comma fields: field 0 unused, 1 = name, 2 = host, 3 = port. The client closes the old connection and logs in again to field 2 and 3 with a new key and the same stored credentials; no listserver step. Fewer than four fields is ignored; swapped host and port fails to connect.	The text a real service sends is unobserved (the lab text came from the open-source listserver reimplementation). 2.22 untested.	`../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`
`PLI_SHOWIMGPLAYER` (24) relay	**Partly settled** (lab)	2.31 sends `GCHAR 9` at login and about every 5.13 s; 2.22 sends it too, but only with another player in the level. The stock server relays any payload unchanged as `PLO_SHOWIMGPLAYER` (32) to the others in the level; injected payloads (synthetic) caused no disconnect and no visible reaction.	What the 9 means; what a receiver does with it; the exact 2.22 condition.	same note
`PLO_BOARDMODIFY2` (186), 2.22 and 2.31	**Settled** for the client reaction (synthetic frames)	Layout `c mapX, c mapY, c x, c y, c w, c h + w*h` GSHORT tiles. Bytes after the tiles are unread padding (a second record there is not applied). One byte short is dropped silently. A block past column 63 is clipped, not rejected. Identical on 2.22 and 2.31.	No server has been seen sending 186. `mapX`/`mapY` other than the current cell untested.	same note; `../../protocol/packet-ids.md`

Item	Status	What is settled	Still open	Where
2.00 listserver, client side	**Partly settled**	Client sends one plain zlib frame `GCHAR 0` + `newmain`, no account or password, then goes straight to the gserver (no second listserver connection). 2.00 accepts a 2.17-style list reply.	The reply format of a real 2.00-era listserver is **unobserved** (the lab reply was synthetic).	same note; `../decomp-findings-2026-10-24-b/04-v2.00-client.md`
Tamper check (PLI 39) derivation	**Settled**	Two chained CRC-32 words over the client's own memory image, each written as 5 bytes in a non-GINT5, unmasked form; a constant per build. 2.31 re-sends every 3.2 s; 2.22 at a gmap crossing; 2.17 once.	Whether any server enforces it.	`08-tampercheck.md`, `../decomp-findings-2026-10-24-b/01-tamper-check-id39.md`, `../../version-profiles/tampercheck-words.tsv`
`PLO_HIDENPCS` (151)	**Settled**	No payload; removes the NPCs with ids 1 to 999 of the active level (id 0 excluded). Wire format unchanged from 2.17.1 to 2.22.	none	`../decomp-findings-2026-10-24-b/02-server-to-client-layouts.md`, `../../protocol/packet-ids.md`
`PLO_SIGNATURE` (25) quit rule	**Settled**	Value 1 quits; value 0 quits unless the connected host string is the loopback address; any other value is ignored. The public-server value 73 is in the ignored group, so it never triggers the quit rule. Same in 2.17.1, 2.22, 2.31; the id is handled from 2.00.	none	`../decomp-findings-2026-10-24-b/05-open-items.md`, table below

Not settled, for contrast (all in `../decomp-findings-2026-10-24-b/05-open-items.md`): `PLI_SHOWIMGPLAYER` (24) payload meaning, the extra 2.31 step after `PLO_SETACTIVELEVEL` (156), client ids 36/42/45 in 2.22, the 2.31 window-title scan.

Still open after the 2026-10-24 decompile pass (new or remaining)

- How other players see a Ctrl+letter emote (topic 11): the key path sends nothing, and the captured SHOWIMG frames are unexplained. A lab capture of a real 2.05 client would settle it.
- Swim on the wire (topic 11, 04): none found in 2.05; swim is a local mode.
- Damage formula (topic 02): inferred sword power / 2, unconfirmed.
- A few unlabelled player-mode values (topics 04, 11). STATUS bit 0x20 is the `hidesword` timer (`../status-bit-0x20.md`).
- The "same name group" rule for the saved respawn point (topic 04).
- 1.39: F7 popup items, death handler and sword hit box were not read (topics 10, 04, 02).
- History window contents and the ignore auto-reply send condition (topic 10).

Tag note: `[dec]` marks a fact derived from the 2026-10-24 static analysis of a client build (behaviour only, no code); `[srv]`, `[cap]`, `[inf]` and version tags keep their earlier meaning (server source, capture, inferred).

Corrections made on 2026-10-24 from the decompile pass: topic 11 no longer says Ctrl+letter sends `PLI_SHOWIMG` (24) (only the script command `showimg` does), and topic 10 no longer lists `TPersonAction` as an unexplained player form. Facts are derived; see `../decomp-findings-2026-10-24/`.

Public-server "unhandled" ids, identified from GServer-v2 and placed

Id	Name	Direction	Meaning in one line	Topic
10	ISLEADER	S to C	you run this level's baddies and timeouts	02, 04 (07 for routing)
25	SIGNATURE	S to C	one byte, 73 = more than eight players; sent at every login. Client rule (2.17–2.31 handlers, 2026-10-24): value 1 quits, value 0 quits unless the host is the loopback address, any other value (such as 73) is ignored, so 73 never triggers the quit rule (settled, see the settled index above)	cross-cutting (below)
39	LEVELMODTIME	S to C	`i5` level modification time for the client cache (not the same as PLI 39)	06
41	STARTMESSAGE	S to C	server message text (HTML, one line)	12
44	HASNPCSERVER	S to C	only with the NPC-server build; the lab build does not send it	07
47	STAFFGUILDS	S to C	guild names that count as staff in the player list	10
156	SETACTIVELEVEL	S to C	level name that receives following NPC/chest/ baddy packets (the gmap name on a gmap)	06
168	UNKNOWN168	S to C	blank packet the login server sends	cross-cutting
171	BIGMAP	S to C	`text,image,x,y` big map settings (2.1+)	06
172	MINIMAP	S to C	`text,image,x,y` minimap settings (2.1+)	06
174	GHOSTICON	S to C	1 = ghost players present (free accounts); GServer sends 0 per level	cross-cutting
179	RPGWINDOW	S to C	two greeting lines in quotes	12
180	STATUSLIST	S to C	player-list status names (Online, Away, ...)	14
182	LISTPROCESSES	S to C	process-list request (anti-cheat), client reply is PLI 44	08

Cross-cutting handling, all one-liners: store 25 and 168 and ignore; store the ghost flag from 174; none of these changes play. Also note `PLI 39` (client to server) is the tampercheck in the Graal_analysis notes and `PLI_MAPINFO` in the older GServer-v2 header (topic 08).

Facts worth knowing before writing code

- **GServer-v2 drops a client after the 6th unknown packet id** (`msgPLI_NULL`). Never send an id that is not in the profile's `pli` set, and never `PLI 43` (mute) to it (topic 14).
- **Most of the game is client-authoritative.** The server clips and forwards. Inventory counts, hearts (not above the server value under 40 AP), death bit and combat events all come from the client (topics 02, 03, 04).
- **Lab server facts**: built without the NPC-server flag; options in `servers/lab/config/serveroptions.txt` (bushitems, respawntime 15, apsystem on, maxplayers 128); `maps`, `gmaps`, `bigmap` and `minimap` are all empty; `servermessage.html` is the stock sample. To test a topic, edit a **copy** of the config and restart the lab gserver; use display :9 only.
- **Privacy**: the pcaps contain the lab password in plaintext. Use the cleaned decoded frames (`examples/lab_replay`) and never commit pcaps or the box-only files.
- **Snapshots**: laundry list "Done" items (warp, other players, DELPLAYER, file transfer) match later Seihai work than the older local clones; these notes use `main` at `06aacfd`.

Rules these notes follow

Specs are derived notes and are untested. GServer-v2 is described, not copied. Graal_analysis boundary: derived, cleaned notes only.